

FUTURESTICK

Next-Generation Job Listing & Career Platform

Complete Technical Documentation, Architecture Blueprint & Implementation Guide

Document Version 1.0

Prepared: 2026

Includes: Frontend, Backend, Database Schemas, AI Modules, Live Interview Engine, Security & Deployment

Table of Contents

(Auto-generated — Right-click and select "Update Field" in Word to refresh)

- Table of Contents..... 2
- 1. Executive Overview..... 9
 - 1.1 Platform Vision..... 9
 - 1.2 Document Scope 9
 - 1.3 Three-Panel Architecture Summary..... 9
- 2. System Architecture..... 11
 - 2.1 High-Level Architecture 11
 - 2.1.1 Tier-by-Tier Breakdown 11
 - 2.2 Why a Hybrid Backend? 11
 - 2.3 Why a Hybrid Database?..... 12
 - 2.4 Architecture Diagram (Textual)..... 12
- 3. Technology Stack 14
 - 3.1 Frontend Stack 14
 - 3.2 Backend Stack — Node.js..... 15
 - 3.3 Backend Stack — Python 16
 - 3.4 Data, Infrastructure, and Tooling..... 17
- 4. Frontend Architecture 18
 - 4.1 Application Topology 18
 - 4.1.1 Why Separate SPAs Instead of One App? 18
 - 4.2 Shared Design System..... 18
 - 4.3 Folder Structure (per panel) 18
 - 4.4 Routing & Code-Splitting..... 18
 - 4.5 State Management Strategy 20
 - 4.5.1 Zustand Slice Example..... 20
 - 4.6 Real-Time Layer..... 20
 - 4.6.1 Socket Channels 20
 - 4.7 Accessibility & Performance 20
- 5. User Panel — Pages and Features 21
 - 5.1 Onboarding & Authentication..... 21
 - 5.1.1 Sign Up Page..... 21
 - 5.1.2 Login Page 21
 - 5.1.3 Forgot Password Flow 21

5.2 Social Feed	21
5.2.1 Feed Home Page (/)	21
Components on this page	22
API endpoints used	22
5.2.2 Post Detail Page (/post/:id).....	22
5.2.3 Looking-for-Job Posts.....	22
5.2.4 Communities	22
5.3 Job Discovery	23
5.3.1 Job Board (/jobs).....	23
Filter rail	23
Sorting options	23
5.3.2 Job Detail Page (/jobs/:id).....	23
Apply flow	24
5.3.3 Saved Jobs (/saved).....	24
5.3.4 Applications (/applications)	24
5.3.5 Competitions (/competitions).....	24
5.4 Profile & Resume	25
5.4.1 My Profile (/profile)	25
Profile sections.....	25
5.4.2 Public Profile (/u/:handle).....	25
5.4.3 Resume Studio (/resume)	25
Template Picker	25
Builder	26
Enhancer	26
Export.....	26
5.5 AI Career Tools	27
5.5.1 AI Career Counselor (/ai/counselor)	27
5.5.2 AI Job Matcher (/ai/matcher)	27
5.5.3 AI Auto-Apply (/ai/auto-apply)	27
5.5.4 AI Interviewer for Practice (/ai/interview-practice)	27
5.5.5 AI Resume Enhancer (/ai/resume-enhancer)	27
5.5.6 AI Cover Letter Generator (/ai/cover-letter)	28
5.5.7 AI Skill Gap Analyzer (/ai/skill-gap)	28
5.5.8 AI Salary Negotiator (/ai/negotiator).....	28
5.6 Interviews.....	28

5.6.1 Interviews Hub (/interviews)	28
5.6.2 Live Interview Room (/interview/:id)	28
5.7 Communication	28
5.7.1 Chat (/chat)	28
5.7.2 Connections (/connections)	29
5.7.3 Notifications (/notifications)	29
5.8 Learning & Career Building	29
5.8.1 Learning Hub (/learning)	29
5.8.2 Mentor Marketplace (/learning/mentors)	29
5.8.3 Career Paths (/learning/paths)	29
5.9 Settings	29
5.9.1 Account Settings (/settings)	29
6. Company Panel — Pages and Features	31
6.1 Onboarding & Verification	31
6.1.1 Company Sign-Up	31
6.1.2 Team Invitations	31
6.2 Dashboard	31
6.2.1 Company Dashboard (/dashboard)	31
6.3 Jobs & Hiring	31
6.3.1 Job Posts List (/jobs)	31
6.3.2 Create / Edit Job (/jobs/new, /jobs/:id/edit)	32
6.3.3 Job Detail View (/jobs/:id)	32
6.3.4 Talent Search (/talent)	32
6.4 Candidate Pipeline	32
6.4.1 Candidate Detail (/candidates/:id)	33
AI Insights on Candidate	33
6.4.2 Bulk Actions	33
6.5 Interviews	33
6.5.1 Interview Calendar (/interviews)	33
6.5.2 Schedule Interview	33
6.5.3 Interview Templates	33
6.5.4 Live Interview Host (/interview/:id)	34
6.5.5 Scorecards	34
6.6 Competitions & Contests	35
6.6.1 Competitions Manager (/competitions)	35

6.6.2 Submissions Review	35
6.7 Company Social & Branding	35
6.7.1 Company Profile (/profile)	35
6.7.2 Posts (/posts)	35
6.7.3 Inbox (/inbox)	35
6.8 Analytics	35
6.8.1 Hiring Analytics (/analytics)	35
6.9 Settings	36
6.9.1 Workspace Settings	36
7. Admin Panel — Pages and Features	37
7.1 Admin Dashboard (/dashboard)	37
7.2 User Management (/users)	37
7.3 Company Management (/companies)	37
7.4 Content Moderation (/moderation)	37
7.5 Jobs Moderation (/jobs)	37
7.6 AI Operations (/ai-ops)	37
7.7 Payments & Billing (/billing)	38
7.8 Trust, Safety & Compliance (/safety)	38
7.9 Platform Configuration (/config)	38
7.10 Observability (/ops)	38
7.11 Audit Log (/audit)	38
8. Backend Architecture	40
8.1 Service Inventory	40
8.2 API Gateway	41
8.2.1 Routing Map	41
8.3 Authentication & Authorization	41
8.3.1 Token Strategy	41
8.3.2 RBAC Model	41
8.3.3 OAuth Providers	41
8.4 Inter-Service Communication	42
8.4.1 Synchronous (HTTP)	42
8.4.2 Asynchronous (Events)	42
8.5 API Endpoint Catalog	43
8.5.1 Auth	43
8.5.2 Users & Profiles	43

8.5.3 Companies.....	43
8.5.4 Jobs	43
8.5.5 Applications.....	43
8.5.6 Social	43
8.5.7 Chat	43
8.5.8 Interviews.....	43
8.5.9 AI	44
9. AI Modules — Detailed Specifications	45
9.1 AI Orchestrator	45
9.2 Resume Parser	45
9.3 Resume Builder	45
9.4 Resume Enhancer	45
9.5 AI Job Matcher	46
9.6 AI Auto-Apply	46
9.7 AI Career Counselor	46
9.8 AI Interviewer (Practice)	46
9.9 AI Cover Letter Generator.....	47
9.10 AI Skill Gap Analyzer.....	47
9.11 AI Salary Negotiator	47
9.12 AI Moderation	47
10. Live Interview Engine	48
10.1 Goals	48
10.2 Media Architecture	48
10.2.1 Signaling Flow	48
10.3 Proctoring Subsystem	48
10.3.1 Proctoring Signals.....	48
10.3.2 Integrity Score	49
10.4 Tools Inside the Room.....	49
10.5 Recording & Storage	50
10.6 AI-Generated Interview Summary	50
10.7 Network Resilience	50
11. Database Schemas	51
11.1 Storage Strategy Recap	51
11.2 MySQL Schemas	51
11.2.1 users	51

11.2.2 user_oauth_identities.....	51
11.2.3 user_profile_sections.....	51
11.2.4 companies	53
11.2.5 jobs.....	53
11.2.6 applications	55
11.2.7 interviews.....	55
11.2.8 competitions	56
11.2.9 payments.....	56
11.2.10 connections, follows, notifications	56
11.3 MongoDB Collections.....	58
11.3.1 posts.....	58
11.3.2 comments	58
11.3.3 reactions.....	58
11.3.4 conversations & messages.....	58
11.3.5 feed_index	58
11.3.6 ai_artifacts	59
11.3.7 interview_events	59
11.3.8 communities	59
11.4 Redis Keyspaces	59
11.5 Elasticsearch Indices	59
12. Security & Compliance.....	61
12.1 Threat Model	61
12.2 Authentication Hardening.....	61
12.3 Authorization	61
12.4 Data Protection	61
12.5 Privacy & Compliance	61
12.6 Application Security	62
12.7 Anti-Abuse.....	62
13. Real-Time Layer & Messaging.....	63
13.1 Socket.IO Topology	63
13.2 Notification Pipeline.....	63
13.3 Chat Delivery.....	63
13.4 Feed Fan-Out.....	63
14. Public Landing Page	64
14.1 Goals	64

14.2 Page Inventory	64
14.3 Folder Structure	64
14.4 Performance Targets.....	64
14.5 SEO	65
15. Infrastructure & Deployment.....	66
15.1 Environments	66
15.2 Cluster Layout	66
15.3 Datastore Topology.....	66
15.4 CI/CD	66
15.5 Observability	67
15.6 Disaster Recovery.....	67
16. Testing Strategy	68
16.1 Test Pyramid	68
16.2 Critical User Journeys (E2E)	68
17. Implementation Roadmap	69
17.1 Phasing Philosophy	69
17.2 Phase 1 — Foundation (Weeks 1-8).....	69
17.3 Phase 2 — Social Layer (Weeks 9-14)	69
17.4 Phase 3 — AI Tools (Weeks 15-22)	69
17.5 Phase 4 — Live Interviews (Weeks 23-30)	69
17.6 Phase 5 — Competitions & Marketplace (Weeks 31-36)	70
17.7 Phase 6 — Scale & Hardening (Weeks 37-44)	70
17.8 Phase 7 — Continuous Enhancement.....	70
18. Appendices.....	71
18.1 Glossary.....	71
18.2 Naming Conventions.....	71
18.3 Versioning	71
18.4 Document Index Summary	72

1. Executive Overview

1.1 Platform Vision

Futurestick is a comprehensive, AI-powered career and recruitment ecosystem engineered to unify the entire hiring lifecycle in a single platform. Unlike traditional job boards that act purely as listing aggregators, Futurestick combines a professional social network, an advanced AI-driven career toolkit, a fully integrated live video interview engine with proctoring, and a competitive hiring environment featuring contests with prizes. The platform is designed to serve three primary actor types — job seekers (users), employers (companies), and platform administrators — each with dedicated panels, distinct role-based access, and tailored workflows.

The system is architected as a polyglot, microservices-oriented application. The frontend uses React with Tailwind CSS for the application panels, while the public landing page is built using vanilla HTML, CSS, and JavaScript for maximum performance and SEO. The backend uses a hybrid runtime: Python (FastAPI) handles all AI-intensive operations such as resume building, job matching, career counseling, and live interview proctoring; Node.js (Express + Socket.IO) handles real-time features such as chat, notifications, video signaling, and the social feed. Persistence is split across MySQL for transactional, relational data (users, companies, jobs, applications, payments) and MongoDB for flexible, document-shaped data (social posts, feeds, chat messages, interview recordings metadata, AI artifacts).

1.2 Document Scope

This document is a complete technical blueprint. It does not contain a business plan, marketing strategy, financial projections, or revenue model. Instead, it provides exhaustive coverage of:

- All user-facing pages and their components across all three panels.
- Complete frontend architecture, routing, state management, and component hierarchy.
- Complete backend architecture, including Python and Node.js service responsibilities, API endpoints, and inter-service communication.
- Complete database schema for both MySQL (relational) and MongoDB (document) stores, including indices and relationships.
- All AI modules with their model choices, pipelines, prompts, and integration patterns.
- The live video interview engine including face tracking, eye tracking, tab-switch detection, multi-person detection, and other proctoring features.
- Security model, role-based access control, authentication, and authorization.
- Real-time communication architecture (chat, notifications, video signaling).
- Deployment architecture, infrastructure, CI/CD, monitoring, and scaling strategy.

1.3 Three-Panel Architecture Summary

Futurestick is organized into three distinct yet interconnected panels, each rendered as a separate React application sharing a common component library and design system:

- **User Panel:** The job-seeker experience. Contains the social feed, job discovery, AI tools (resume builder, career counselor, interviewer, job matcher, auto-apply), profile, applications, interviews, chat, connections, and learning paths.
- **Company Panel:** The employer experience. Contains job posting, application management, candidate pipeline, interview scheduling, live interview hosting, contest creation, hiring analytics, company social posts, and HR team management.
- **Admin Panel:** The platform operator experience. Contains user moderation, company verification, content moderation, payment management, AI usage analytics, system health, role management, and platform-wide configuration.

Role-based access control (RBAC) is enforced at three layers: at the API gateway via JWT claims, at the service layer via decorators on each route, and at the database layer via row-level access policies. Every endpoint declares its required role(s) and permissions, and the frontend dynamically renders only what the current user is authorized to see.

2. System Architecture

2.1 High-Level Architecture

Futurestick uses a layered, service-oriented architecture composed of seven primary tiers. Requests originate from one of four client surfaces — the public landing page, the user panel, the company panel, or the admin panel — and flow through a CDN-fronted load balancer into the API gateway. The gateway authenticates the request, applies rate limiting and request shaping, and routes the call to the appropriate backend service. Backend services are split between Node.js (real-time, transactional, social) and Python (AI, computer vision, machine learning). Both runtimes share the data layer, which is a hybrid of MySQL, MongoDB, Redis (caching and pub/sub), and an object store (S3-compatible) for media.

2.1.1 Tier-by-Tier Breakdown

- **Tier 1 — Client:** Four distinct client applications: the marketing/landing site (HTML/CSS/JS), and three React + Tailwind single-page applications for users, companies, and admins. Mobile apps can later wrap these via React Native or be served as PWAs.
- **Tier 2 — Edge / CDN:** CloudFront or Cloudflare for static assets, image transformations, and DDoS mitigation. The landing site is fully static and served from the edge. The React apps are also edge-cached but hydrate against APIs.
- **Tier 3 — API Gateway:** An NGINX or Kong-based gateway that terminates TLS, validates JWT tokens, applies rate limiting, performs request logging, and routes traffic. The gateway also handles WebSocket upgrades for real-time channels.
- **Tier 4 — Service Layer (Node.js):** Auth service, user service, company service, job service, application service, social/feed service, chat service, notification service, interview signaling service, payment service. Built with Express and Socket.IO.
- **Tier 5 — Service Layer (Python):** AI orchestration service, resume builder, resume enhancer, AI interviewer, AI career counselor, job matcher, auto-apply engine, and the live interview proctoring engine (face tracking, eye tracking, multi-person detection). Built with FastAPI, Celery, and PyTorch/MediaPipe/OpenCV.
- **Tier 6 — Data Layer:** MySQL 8 (transactional), MongoDB 7 (document), Redis 7 (cache + pub/sub + sessions), Elasticsearch (search), S3 (media).
- **Tier 7 — Background Workers:** Celery (Python) workers for AI tasks; BullMQ (Node) workers for emails, notifications, scheduled jobs, and webhooks.

2.2 Why a Hybrid Backend?

A single backend runtime would force compromises. Node.js excels at concurrent I/O-bound workloads — chat fan-out, WebRTC signaling, social feed delivery, real-time notifications — but is suboptimal for CPU-bound machine learning. Python has the richest AI ecosystem (PyTorch, Hugging Face, MediaPipe,

OpenCV, LangChain) but is slower for high-concurrency I/O. By splitting responsibilities along workload type, Futurestick can scale each tier independently and use the best tool for each job.

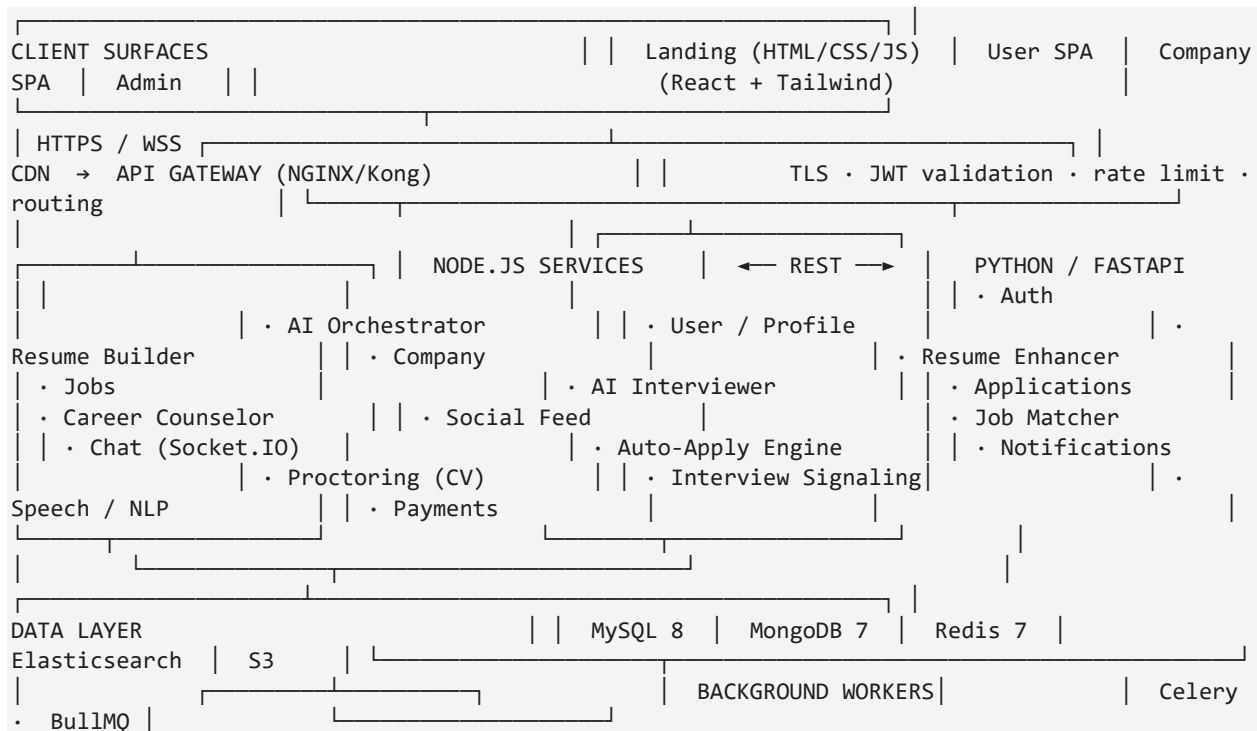
Inter-service communication uses two patterns. For synchronous calls (e.g., the user service requesting a resume score from the AI service), services communicate over internal HTTP with mTLS. For asynchronous workflows (e.g., enqueueing an AI resume generation job, or fanning out a new social post to followers), services use Redis Streams or RabbitMQ as the message bus.

2.3 Why a Hybrid Database?

Relational integrity matters for entities like users, companies, jobs, applications, payments, and interview schedules. These have strict schemas, foreign-key relationships, and require ACID guarantees — MySQL is the right fit. However, many features have variable, evolving, or hierarchical shapes: social posts with arbitrary embedded media, comments and nested replies, chat message threads, AI interaction history, generated resume documents, interview event logs, and feed personalization data. Forcing these into rigid tables leads to schema sprawl and join-heavy queries. MongoDB stores them as documents naturally.

The two databases are not duplicated; they are complementary. A user's identity and authentication live in MySQL; their social feed posts and chat history live in MongoDB. Cross-store consistency is maintained via an outbox pattern: writes to MySQL emit domain events to a Kafka or Redis stream, which a worker consumes and applies to MongoDB.

2.4 Architecture Diagram (Textual)



3. Technology Stack

3.1 Frontend Stack

Layer	Technology	Purpose / Justification
Framework	React 18 + Vite	Concurrent rendering, fast HMR, modern build pipeline
Styling	Tailwind CSS 3.4	Utility-first, design-token driven, small final bundles
Routing	React Router v6	Nested routes, data loaders, role-based guards
State (Server)	TanStack Query v5	Caching, deduping, optimistic updates
State (Client)	Zustand + Context	Lightweight global state, slice-based stores
Forms	React Hook Form + Zod	Schema-validated, performant, type-safe
Realtime	Socket.IO Client	Chat, feed updates, notifications, interview signaling
Video / WebRTC	simple-peer + mediasoup-client	Scalable SFU-based video
Charts	Recharts + visx	Analytics dashboards
Editor	TipTap (ProseMirror)	Rich-text for posts, resume sections, messages
Animation	Framer Motion	Page transitions, micro-interactions
Icons	Lucide React	Consistent open-source icon set
i18n	react-i18next	Multi-language support
Testing	Vitest + RTL + Playwright	Unit, component, and end-to-end testing
Landing Page	HTML5 + CSS3 + Vanilla JS + GSAP	Static, SEO-optimized, fast-loading

3.2 Backend Stack — Node.js

Layer	Technology	Purpose / Justification
Runtime	Node.js 20 LTS	Long-term support, V8 performance
Framework	Express.js 4 / Fastify	Mature, large ecosystem
Realtime	Socket.IO 4 + Redis adapter	Horizontal scaling of WebSockets
ORM (MySQL)	Prisma 5	Type-safe queries, migrations, generated types
ODM (MongoDB)	Mongoose 8	Schema validation for documents
Auth	Passport.js + JWT + bcrypt	Local, OAuth (Google, LinkedIn, GitHub)
Validation	Zod / Joi	Request and DTO validation
Queue	BullMQ + Redis	Email, notification, scheduled jobs
File Upload	Multer + S3 SDK	Direct-to-S3 multipart uploads
Email	Nodemailer + SES	Transactional email
Logging	Pino + OpenTelemetry	Structured JSON logs, distributed tracing
Testing	Jest + Supertest	Unit + integration tests

3.3 Backend Stack — Python

Layer	Technology	Purpose / Justification
Runtime	Python 3.12	Modern syntax, performance
Framework	FastAPI	Async, OpenAPI auto-generation, Pydantic validation
ASGI Server	Uvicorn + Gunicorn	Production-grade async server
Task Queue	Celery + Redis broker	Long-running AI jobs
LLM Orchestration	LangChain + LangGraph	Multi-step AI flows, agent orchestration
LLM Providers	OpenAI / Anthropic / open-source via Ollama	Configurable per-feature
Embeddings	sentence-transformers + pgvector / Pinecone	Semantic search, job matching
Computer Vision	MediaPipe + OpenCV + dlib	Face mesh, eye tracking, head pose
ML Models	PyTorch + ONNX Runtime	Custom proctoring models, fast inference
Speech	Whisper + Coqui TTS	Interview transcription, AI voice
PDF Generation	WeasyPrint + ReportLab	Resume PDF rendering
Validation	Pydantic v2	Runtime data validation
DB Drivers	SQLAlchemy 2 (async) + Motor	MySQL + MongoDB async access
Testing	Pytest + httpx	Async test client

3.4 Data, Infrastructure, and Tooling

Layer	Technology	Purpose / Justification
Relational DB	MySQL 8	ACID, transactional core entities
Document DB	MongoDB 7	Posts, chat, AI artifacts, flexible schemas
Cache / Pub-Sub	Redis 7	Session, cache, queue broker, Socket.IO adapter
Search	Elasticsearch / OpenSearch	Full-text job + people search
Object Storage	AWS S3 / MinIO	Resumes, videos, images, recordings
CDN	CloudFront / Cloudflare	Static asset and media delivery
Containerization	Docker + Docker Compose	Local dev parity
Orchestration	Kubernetes (EKS/GKE)	Production scaling
IaC	Terraform	Reproducible infrastructure
CI/CD	GitHub Actions + ArgoCD	Automated build + GitOps deploy
Monitoring	Prometheus + Grafana + Loki	Metrics + logs
APM	Sentry + OpenTelemetry	Error tracking, distributed traces
Secrets	HashiCorp Vault / AWS Secrets Manager	Encrypted secret storage
TURN/STUN	coturn	WebRTC NAT traversal for interviews
SFU	mediasoup / LiveKit	Scalable multi-party video

4. Frontend Architecture

4.1 Application Topology

Futurestick's frontend is split into four independent deployable units, each tuned for its purpose. The landing page is a static HTML/CSS/JS site optimized for first-paint and SEO. The three React panels (user, company, admin) share a common design system and component library, but are deployed as separate single-page applications behind subdomains: `app.futurestick.com` (user), `business.futurestick.com` (company), and `admin.futurestick.com` (admin). Each panel boots independently, ships only the code its actor needs, and reaches the same backend API.

4.1.1 Why Separate SPAs Instead of One App?

- **Bundle Size:** Admins do not need the social feed code; users do not need the candidate-pipeline code. Splitting cuts each bundle by 40-60%.
- **Security Posture:** Admin code never ships to user browsers, reducing the surface area for inspection and reverse-engineering of moderation logic.
- **Independent Deployment:** Each panel can be released, rolled back, or A/B tested in isolation.
- **Routing Clarity:** Each domain has its own auth flow and route tree, preventing accidental cross-panel link leakage.

4.2 Shared Design System

All three panels consume a shared package, `@futurestick/ui`, which exports atomic components (Button, Input, Modal, Card, Avatar, Badge, Toast), composite components (DataTable, FormBuilder, FilePicker, RichTextEditor, VideoTile, Chart), and design tokens (colors, spacing, typography, shadows, motion). The package is built with Tailwind's preset feature so each consumer extends it; tokens are defined in a single `tailwind.preset.js` file. The library uses Storybook for component documentation and ships its own visual regression test suite using Chromatic.

4.3 Folder Structure (per panel)

```
apps/user-panel/ | public/ | src/ | | api/ | | | | # Axios + TanStack Query
hooks | | | client.ts | | | jobs.ts | | | posts.ts | | | interviews.ts
| | | ai.ts | | | assets/ | | | components/ | | | feed/ | | | jobs/ |
| | | profile/ | | | interview/ | | | ai/ | | | chat/ | | | features/
# Feature slices (state + UI) | | | auth/ | | | feed/ | | | jobs/ | | |
| applications/ | | | interviews/ | | | resume/ | | | chat/ | | |
connections/ | | | hooks/ | | | layouts/ | | | pages/ | | | # Route components |
| routes/ | | | index.tsx | | | guards.tsx | | | # Auth + role guards | | | | |
| lazy.ts | | | stores/ | | | # Zustand slices | | | lib/ | | | socket.ts |
| | webrtc.ts | | | proctor.ts | | | styles/ | | | utils/ | | | App.tsx |
| main.tsx | | index.html | | tailwind.config.js | | vite.config.ts | | package.json
```

4.4 Routing & Code-Splitting

Each panel uses React Router v6 with lazy-loaded route segments. Top-level layouts handle auth, role checks, and shared chrome (header, sidebar, footer). Route segments are split per feature, ensuring the initial bundle stays under 250 KB gzipped. Suspense boundaries wrap each lazy route with a skeleton loader matching the destination's layout.

```
// routes/index.tsx (User Panel)
const Feed = lazy(() => import('@pages/Feed'));
const JobBoard = lazy(() => import('@pages/JobBoard'));
const JobDetail = lazy(() => import('@pages/JobDetail'));
const Profile = lazy(() => import('@pages/Profile'));
const ResumeStudio = lazy(() => import('@pages/ResumeStudio'));
const Interview = lazy(() => import('@pages/Interview'));
const Chat = lazy(() => import('@pages/Chat'));
export const router = createBrowserRouter([
  { path: '/login', element: <Login /> },
  { path: '/signup', element: <Signup /> },
  { path: '/', element: <Feed /> },
  { path: '/jobs', element: <JobBoard /> },
  { path: '/jobs/:id', element: <JobDetail /> },
  { path: '/profile/:handle?', element: <Profile /> },
  { path: '/resume', element: <ResumeStudio /> },
  { path: '/ai/*', element: <AIToolsRoutes /> },
  { path: '/interview/:id', element: <Interview /> },
  { path: '/chat', element: <Chat /> },
  { path: '/applications', element: <Applications /> },
  { path: '/competitions', element: <Competitions /> },
  { path: '/learning', element: <Learning /> },
] ]]);
```

4.5 State Management Strategy

State in Futurestick is divided into four categories, each handled by the most appropriate tool. Server state is cached in TanStack Query, which deduplicates requests, refetches on window focus, and provides optimistic update primitives. Client UI state (modals open, sidebar collapsed, theme, draft messages) lives in Zustand stores split by feature slice. Form state is owned by React Hook Form, with Zod schemas used for validation and TypeScript inference. URL state — filters, pagination cursors, current tab — lives in the URL via search params, making every view shareable and bookmarkable.

4.5.1 Zustand Slice Example

```
// stores/feed.store.ts import { create } from 'zustand'; import { devtools, persist } from
'zustand/middleware'; interface FeedState { composerOpen: boolean; draft: string;
filter: 'all' | 'connections' | 'companies' | 'competitions'; setComposerOpen: (open:
boolean) => void; setDraft: (text: string) => void; setFilter: (filter:
FeedState['filter']) => void; } export const useFeedStore = create<FeedState>()(
devtools(persist( (set) => ({ composerOpen: false, draft: '', filter:
'all', setComposerOpen: (composerOpen) => set({ composerOpen }), setDraft: (draft)
=> set({ draft }), setFilter: (filter) => set({ filter }), }, { name: 'feed-
store', partialize: (s) => ({ draft: s.draft, filter: s.filter }) } )) );
```

4.6 Real-Time Layer

Real-time data is delivered through Socket.IO. A single client-side socket connects on app boot after authentication, then joins multiple rooms based on the user's identity, conversations, and active interview. Events are dispatched through a typed event bus that integrates with TanStack Query's invalidation system, so receiving a new chat message both updates UI immediately and invalidates the relevant query keys.

4.6.1 Socket Channels

- **user:{userId}**: Personal channel for notifications, follow events, application status changes
- **conversation:{id}**: Per-chat-thread real-time messaging
- **post:{id}**: Live reactions and comments on a specific post
- **feed:{userId}**: Personalized feed delta updates
- **interview:{id}**: Interview signaling, proctoring events, participant join/leave
- **company:{id}**: Company-wide events for HR teams

4.7 Accessibility & Performance

Every component in the design system is built to WCAG 2.1 AA. Keyboard navigation, focus rings, ARIA roles, and color contrast are tested on every PR via axe-core in CI. Performance budgets are enforced: 250 KB initial JS, 50 KB initial CSS, LCP under 2.5 seconds on a simulated 3G Moto G4. Images are served via a thumbnail service with WebP/AVIF negotiation. Lists virtualize past 50 items using react-virtual.

5. User Panel — Pages and Features

The user panel is the primary surface for job seekers. It is structured around six pillars: Social Feed, Job Discovery, AI Career Tools, Profile and Resume, Interviews and Applications, and Communication. Each pillar contains multiple pages, components, and flows. This section covers every page, the components on it, the user flows it supports, and the API calls it makes.

5.1 Onboarding & Authentication

5.1.1 Sign Up Page

New users land on a multi-step onboarding flow that collects credentials, personal information, and initial career data so the platform can immediately personalize the experience. The flow uses React Hook Form with a Zod schema spanning all steps; data is persisted in session storage so refreshes do not lose progress.

- **Step 1 — Account:** Email, password (with strength meter), agree to terms. Optional OAuth: Google, LinkedIn, GitHub.
- **Step 2 — Identity:** Full name, headline, current location, phone (optional), date of birth, gender (optional).
- **Step 3 — Career Snapshot:** Current status (student, working, looking, freelancing), years of experience, current role, current company, industry.
- **Step 4 — Skills & Interests:** Tag-based skill picker with autosuggest from a curated 12,000-skill taxonomy; interest topics for feed personalization.
- **Step 5 — Resume Import:** Upload existing resume (PDF/DOCX) — server runs the resume parser and prefills the entire profile, or skip and start blank.
- **Step 6 — Verify Email:** OTP-based email verification before first login.

5.1.2 Login Page

Standard email/password plus OAuth providers. Includes a passwordless magic-link option for users who forgot their password. Login attempts are rate-limited at the API gateway: five attempts per IP per 15 minutes, then a CAPTCHA challenge. After successful login, the server returns a short-lived access token (15 min) and a long-lived refresh token (30 days, HTTP-only cookie). Two-factor authentication via TOTP or SMS is offered after login if the account has 2FA enabled.

5.1.3 Forgot Password Flow

Three-step flow: enter email, receive 6-digit code, set new password. The reset token is single-use and expires in 15 minutes. The user is logged out of all other sessions on success.

5.2 Social Feed

5.2.1 Feed Home Page (/)

The feed is the user's default landing page after login and the social heartbeat of Futurestick. It displays a chronologically and algorithmically blended timeline of posts from people the user follows, companies they follow, communities they have joined, recommended posts based on their interests, sponsored job posts, and active competitions. The page layout is a three-column shell on desktop: left rail (navigation and quick links), center column (feed), right rail (suggestions, trending tags, upcoming interviews).

Components on this page

- **PostComposer:** Top of feed. Tabs: "Looking for a Job", "Share an Update", "Ask the Community", "Showcase Work". Supports text, images, video, polls, link previews, code blocks, and tagged people/companies.
- **FeedFilter:** Tabs to filter: All, Connections, Companies, Competitions, Communities. Persists to URL via search params.
- **PostCard:** Renders any post type. Shows author avatar/name/headline, timestamp, content, media gallery, reaction summary, comment count, share count, and a kebab menu (save, hide, report, mute author, copy link).
- **ReactionBar:** Like, Celebrate, Insightful, Support, Funny, Curious — six reactions with hover-to-pick like LinkedIn.
- **CommentThread:** Nested comments up to two levels deep, with @mentions, reactions, and real-time append via Socket.IO.
- **RightRailWidgets:** Suggested connections, trending tags, upcoming interviews, active competitions, learning recommendations.

API endpoints used

```
GET /api/v1/feed?cursor=<cursor>&filter=<filter> POST /api/v1/posts POST
/api/v1/posts/:id/reactions POST /api/v1/posts/:id/comments GET /api/v1/feed/suggestions WS
feed:{userId} (real-time post insertions and reaction updates)
```

5.2.2 Post Detail Page (/post/:id)

Full-page view of a single post. Used for deep linking and SEO. Includes the full post body without truncation, all comments expanded, related posts, and a sticky author card with a follow/connect button. Comments support pagination with "Load older comments" both above and below the visible window.

5.2.3 Looking-for-Job Posts

A specialized post type. When a user toggles the "Looking for a Job" tab in the composer, the post is structured: target role, locations, work mode (remote/hybrid/onsite), notice period, expected compensation range (optional), tagged skills, and a free-text pitch. These posts appear in a separate "Talent Wanted" feed visible to companies, and recruiters can directly message or invite the candidate to apply with one click.

5.2.4 Communities

Topic-based mini-feeds. Users can join communities such as "Frontend Engineers India", "Data Science Bootcamp Alumni", "Women in Tech", "Remote Work". Each community has its own feed, members, moderators, rules, and pinned posts. Communities are created by users (subject to admin approval) and can be public, private (request to join), or invitation-only.

5.3 Job Discovery

5.3.1 Job Board (/jobs)

The job board is the primary marketplace surface. It displays an infinite-scroll list of jobs with a powerful filter rail. Each job card shows the company logo, title, location, work mode, salary band (if disclosed), key skills, and a one-tap "Save" and "Apply" button. Clicking a card opens the job detail in a side panel on desktop or full screen on mobile, preserving scroll position in the list.

Filter rail

- Free-text search (matches title, description, skills, company)
- Location (with radius slider) or Remote-only toggle
- Work mode: Remote, Hybrid, Onsite
- Employment type: Full-time, Part-time, Contract, Internship, Freelance
- Experience: Entry, Mid, Senior, Lead, Executive (and exact years range)
- Salary range slider (currency-aware)
- Skills (multi-select with autocomplete from skill taxonomy)
- Industry / Function
- Company size: Startup, SMB, Enterprise, Public
- Posted within: 24h, 7d, 14d, 30d
- Diversity-friendly, Visa-sponsored, Has Equity, 4-day Week

Sorting options

- Best Match (AI-scored relevance to user profile)
- Most Recent
- Highest Salary
- Most Applied (popularity)
- Easiest to Apply (one-click eligible)

5.3.2 Job Detail Page (/jobs/:id)

A single job's full detail. Layout: hero with company logo, title, location, posted-by avatar; tabs for Overview, Requirements, Benefits, Company, Process, Similar Jobs, Reviews. The Apply CTA is sticky on scroll. The page also surfaces the AI Match Score (0-100), explaining why this job fits the user via four sub-scores: Skills, Experience, Location, Compensation.

Apply flow

- One-click apply with default resume + cover letter
- Custom apply with attached portfolio, cover letter, and answers to screening questions
- AI-suggested resume tailoring ("Boost your match score from 72 to 89 by emphasizing X")
- After-apply confirmation with timeline of next steps

5.3.3 Saved Jobs (/saved)

All jobs the user has bookmarked. Grouped by status: "Open", "Closing Soon", "Closed". Each card shows when it was saved, the current match score (recomputed daily as the user's profile evolves), and quick apply.

5.3.4 Applications (/applications)

A Kanban-style tracker of every job the user has applied to. Columns: Applied, Under Review, Interview Scheduled, Offer, Rejected, Withdrawn. Each card shows the company, role, applied date, current stage, next action (e.g., "Interview on March 12 at 4 PM"), and a chat button to message the recruiter. Drag-and-drop is disabled — stages are driven by the company panel and the timeline is read-only for the user, except for "Withdraw".

5.3.5 Competitions (/competitions)

A discovery surface for hiring competitions and contests. Companies post coding challenges, design contests, case studies, and hackathons with monetary or in-kind prizes. The user can browse, register, view leaderboards, submit entries, and chat with organizers. Each competition has its own dedicated page (/competitions/:id) with rules, prize pool, schedule, judges, leaderboard, and submission portal.

5.4 Profile & Resume

5.4.1 My Profile (/profile)

The user's public-facing identity on Futurestick. Modeled as a layered, section-rich profile inspired by LinkedIn and personal websites. It is also the canonical source of truth for the AI resume builder — every field on the profile flows into the resume.

Profile sections

- **Hero:** Cover photo, avatar, name, headline, location, pronouns, contact info, social links, follow/connect button. Public visibility toggles per field.
- **About:** Rich-text bio up to 2,000 characters. Markdown rendered.
- **Experience:** List of roles with company, title, dates, location, work mode, employment type, description, skills used, achievements, media.
- **Education:** Institutions, degrees, fields of study, grades, dates, activities, projects.
- **Skills:** Endorsable skill tags with proficiency level and years of experience. Top 3 are featured.
- **Certifications:** Issuer, name, issue date, expiry, credential ID, verification URL.
- **Projects:** Title, description, role, technologies, links (live, repo, video), media gallery, collaborators.
- **Publications & Patents:** Standard academic format.
- **Awards:** Title, issuer, date, description.
- **Languages:** Language and proficiency (Native, Fluent, Professional, Conversational, Basic).
- **Volunteer Work:** Cause, organization, role, dates, description.
- **Recommendations:** Written endorsements from connections, with mutual consent.
- **Activity:** Recent posts, reactions, comments — visible to followers.
- **Career Goals:** Private to user; feeds the AI matcher and counselor.

5.4.2 Public Profile (/u/:handle)

How others see a user. Honors privacy settings: blocked sections are hidden, contact info is gated behind a connection or a paid recruiter view. Includes a "Save to Talent Pool" CTA for company users.

5.4.3 Resume Studio (/resume)

The AI-powered resume builder. Houses three modes: Template Picker, Builder, and Enhancer.

Template Picker

Library of 100+ pre-built templates organized by industry (Tech, Design, Marketing, Finance, Healthcare, Academia, Creative, Executive), style (Modern, Classic, Minimalist, Bold, Creative, ATS-Optimized), and length (1-page, 2-page, multi-page). Each template is filterable, previewable in real time, and reversible — the user can switch templates anytime without losing content. Templates are stored as JSX renderers

so they support dynamic data; printable output is generated server-side via WeasyPrint to ensure pixel-perfect PDFs.

Builder

Two-pane editor. Left: structured form with sections matching the profile model. Right: live preview that re-renders on every keystroke (debounced 300 ms). Sections can be reordered, hidden, or duplicated. The builder offers AI assistance at every text field: "Improve", "Make More Concise", "Add Metrics", "Match this Job Description". The user can have multiple resume variants (e.g., "Backend Engineer", "Engineering Manager"), each tied to different target roles.

Enhancer

Drop in any existing resume (PDF/DOCX) and the AI parses it, scores it on 12 dimensions (clarity, impact, ATS-compatibility, action verbs, quantification, keyword density, readability, length, formatting, consistency, grammar, spelling), and produces a side-by-side enhanced version. Each suggested change is approvable individually.

Export

- PDF (server-rendered, ATS-safe)
- DOCX (editable, mirrors PDF)
- Plain text (for ATS pasting)
- HTML (shareable link with custom domain)
- JSON Resume schema export

5.5 AI Career Tools

5.5.1 AI Career Counselor (/ai/counselor)

A conversational coach that helps the user plan their career. The interface is a chat-first experience with structured side panels for action items, recommended skills, and learning paths. The counselor draws on the user's full profile, target roles, recent applications, and a knowledge graph of careers, skills, and transitions. It can answer questions like "How do I move from QA to ML engineering?", "What roles can I apply for with my current skills?", "What should I learn next to earn 30% more?", and "Should I take this offer?" with reasoning grounded in market data.

5.5.2 AI Job Matcher (/ai/matcher)

A daily-refreshed personalized list of the top 20 jobs matched to the user's profile, with detailed score breakdowns and explanations. The matcher uses a hybrid approach: a sentence-transformer embedding of the user's profile and each job is computed; cosine similarity gives a baseline score; this is then refined by a learned reranker that incorporates explicit signals (location preferences, salary expectations, work mode) and historical click/apply behavior. Matches are explained in plain English: "Strong match because you have 4 years of React experience and the role requires 3+; gap on Kubernetes which is preferred but not required."

5.5.3 AI Auto-Apply (/ai/auto-apply)

A consent-based agent that, after the user defines their preferences and approves a daily quota, automatically applies to matched jobs above a configurable score threshold. Each application uses the user's tailored resume for the role (generated on-the-fly), an AI-written cover letter customized to the job description, and answers to screening questions inferred from the user's profile. The user reviews and approves each application before submission by default; advanced users can enable full autonomy with an audit trail. A daily summary email lists every job applied to.

5.5.4 AI Interviewer for Practice (/ai/interview-practice)

A simulated interview environment where the user can practice with an AI interviewer that adapts to their target role and seniority. Modes: Behavioral, Technical (DSA, system design, role-specific), HR/Cultural Fit, Mixed. The interviewer asks one question at a time, listens to the user's spoken or typed answer, asks follow-ups, and produces a detailed scorecard at the end covering content, structure (STAR adherence), confidence, clarity, vocabulary, pacing, and filler words. Sessions are recorded (with consent) for replay and self-review. The user can choose interviewer persona (friendly, neutral, tough), industry context, and difficulty.

5.5.5 AI Resume Enhancer (/ai/resume-enhancer)

Standalone tool. The user uploads any resume (their own or a competitor's for benchmarking), specifies a target job description (optional), and receives a comprehensive analysis: ATS compatibility score,

keyword match, missing keywords, weak phrases, redundant content, and concrete rewrites. Every suggestion is one click to accept. See section 5.4.3 for deeper detail.

5.5.6 AI Cover Letter Generator (/ai/cover-letter)

Generates tailored cover letters from the user's profile and a job posting. Supports tones (formal, conversational, enthusiastic, technical), lengths (short, standard, detailed), and structures (problem-solution, story-led, achievement-driven). Output is fully editable.

5.5.7 AI Skill Gap Analyzer (/ai/skill-gap)

Compares the user's current skills against the requirements of their target roles (or a specific job) and produces a prioritized learning roadmap. For each skill gap it suggests specific courses (from partner institutes), practice projects, mentors, and a realistic timeline. Integrates with the Learning section to enroll the user directly.

5.5.8 AI Salary Negotiator (/ai/negotiator)

Given an offer, the user's market position, and target compensation, the tool drafts a negotiation script, predicts the company's likely counter, and rehearses the conversation via voice. Includes benchmark data sourced from the platform's anonymized compensation database.

5.6 Interviews

5.6.1 Interviews Hub (/interviews)

Lists all upcoming, in-progress, and past interviews. Each entry shows the company, role, interviewer(s), scheduled time with countdown, format (video, phone, in-person, take-home), join button (active 10 minutes before start), and a preparation checklist generated by AI for that specific interview based on the role and the user's gaps.

5.6.2 Live Interview Room (/interview/:id)

The live video interview surface. See Section 9 for the full proctored interview engine. The user joins through a single button; the system pre-flights camera, microphone, network, and consent for proctoring before admitting them to the room. The room shows the interviewer's video, the user's self-view, a shared whiteboard, a code editor (for technical rounds), a chat panel, and a bank of pre-loaded questions if the interviewer chose to share them. A persistent indicator shows the user that proctoring is active and what is being measured.

5.7 Communication

5.7.1 Chat (/chat)

End-to-end chat for user-to-user, user-to-recruiter, and group conversations. Familiar two-pane layout: conversation list left, active conversation right. Features: typing indicators, read receipts, reactions, file

attachments (up to 25 MB), voice notes, video calls (one-tap escalation), message search, pinned messages, archive, mute, and block. Messages are encrypted in transit and at rest; group keys rotate on membership changes.

5.7.2 Connections (/connections)

Manage the user's social graph. Tabs: Connections (mutual), Following (one-way), Followers, Pending Requests, Suggestions, Blocked. Bulk actions for accepting/declining requests. Suggestions are powered by graph proximity (people-you-may-know) and shared affiliations (school, company, community).

5.7.3 Notifications (/notifications)

Centralized notification inbox. Categories: Jobs (matches, status updates), Social (reactions, comments, mentions, follows), Messages (chat previews), Interviews (reminders, schedule changes), Competitions (results, deadlines), Platform (announcements, security). Each notification deep-links to its source. The user controls which categories trigger email, push, or in-app delivery via /settings/notifications.

5.8 Learning & Career Building

5.8.1 Learning Hub (/learning)

A curated, AI-personalized catalog of courses, micro-credentials, bootcamps, and certifications offered through partnerships with top institutes (universities, MOOCs, online academies). The page features a personalized track based on the user's career goals and skill gaps. Courses are filterable by skill, level, duration, price, provider, and whether they include a verified credential. Completed courses automatically appear on the user's profile and resume.

5.8.2 Mentor Marketplace (/learning/mentors)

Browse industry mentors by domain, role, company, language, and price. Each mentor profile shows their experience, ratings, reviews, available time slots, session formats (1-on-1 video, async chat, mock interview), and packages. Booking is integrated with the platform's calendar; sessions can run inside the same live video room used for interviews.

5.8.3 Career Paths (/learning/paths)

Curated, multi-month learning roadmaps for popular career transitions: "Become a Frontend Engineer", "Switch to Product Management", "Move into Data Science", etc. Each path is a sequence of courses, projects, mentor sessions, and milestone interviews. Progress is tracked and shareable.

5.9 Settings

5.9.1 Account Settings (/settings)

- **Profile:** Edit core identity, headline, location, contact info.
- **Privacy:** Profile visibility, who can find you, who can message you, search-engine indexing.

- **Notifications:** Email, push, in-app preferences per category.
- **Security:** Password, 2FA, active sessions, login history, connected OAuth accounts.
- **Job Preferences:** Open-to-work toggle (visible to recruiters), target roles, locations, salary, work mode, notice period.
- **AI Preferences:** Allowed/blocked AI tools, data-usage consent, auto-apply settings.
- **Billing:** Subscription plan, payment methods, invoices.
- **Data & Privacy:** Download all data, delete account.

6. Company Panel — Pages and Features

The company panel is the employer surface, served at business.futurestick.com. It supports multi-user companies with role-based access: Owner, Admin, Hiring Manager, Recruiter, Interviewer, and Read-Only. Each role has a curated permission set governing which jobs they can edit, which candidates they can see, whether they can host interviews, and whether they can post on behalf of the company. The panel is structured around five pillars: Jobs and Hiring, Candidate Pipeline, Interviews, Social and Branding, and Analytics.

6.1 Onboarding & Verification

6.1.1 Company Sign-Up

A multi-step flow that collects company identity, verifies legitimacy, and provisions the workspace. Steps: (1) Account creation by founding admin, (2) Company details — legal name, brand name, website, industry, size, headquarters, founded year, (3) Verification documents — incorporation certificate, GST/tax ID, authorized signatory ID. Verification is reviewed by platform admins within 24-48 hours. Until verified, the company can configure its workspace but cannot post jobs publicly.

6.1.2 Team Invitations

Owners and admins invite teammates by email, assigning them a role at invitation time. Invites expire in 7 days and can include a custom welcome note. Invitees create their account or, if they already have a Futurestick user account, link it to the company workspace.

6.2 Dashboard

6.2.1 Company Dashboard (/dashboard)

The default landing page after login. Shows top-of-funnel metrics, action items, and a feed of recent activity. Components include a metrics ribbon (active jobs, total applicants this week, interviews scheduled today, offers extended this month), a hiring pipeline funnel visualization, a list of jobs with stalled candidates needing action, recent applicant submissions, upcoming interviews for the logged-in user, and a team-activity feed.

6.3 Jobs & Hiring

6.3.1 Job Posts List (/jobs)

All jobs the company has posted, filterable by status (Draft, Active, Paused, Closed), department, location, and owner. Each row shows title, department, applicants count, days posted, status, owner, and quick actions (edit, duplicate, pause, close, share). Bulk actions let recruiters change status, reassign owner, or export to CSV.

6.3.2 Create / Edit Job (/jobs/new, /jobs/:id/edit)

A guided, AI-assisted job creation flow. Steps and sections:

- **Basics:** Title, department, employment type, work mode, location(s), seniority level, headcount.
- **Description:** Rich-text editor with AI assist ("Generate a description", "Make more inclusive", "Optimize for keywords"). Sections for Responsibilities, Requirements, Nice-to-Haves, Benefits.
- **Skills & Tags:** Auto-suggested from description; recruiter refines.
- **Compensation:** Salary range (with currency), equity, bonus, benefits checklist. Can be hidden or shown publicly.
- **Screening Questions:** Customizable knockout and qualifying questions. AI suggests questions based on the role.
- **Application Form:** Choose required fields: resume, cover letter, portfolio, custom uploads, video pitch.
- **Pipeline Stages:** Default pipeline (Applied → Screen → Interview → Offer → Hired) is editable per job. Stages can include automation (auto-reject after N days, auto-advance on assessment pass).
- **Hiring Team:** Assign owner, recruiters, hiring managers, and interviewers per stage.
- **Visibility & Promotion:** Public, internal-only, or shared via private link. Promote (paid) for top placement and cross-platform syndication.
- **Compliance:** EEOC questions toggle, GDPR data retention period, anonymized review mode.

6.3.3 Job Detail View (/jobs/:id)

A job's full operational view inside the company panel. Tabs:

- **Overview:** Posting preview as candidates see it, plus key metrics.
- **Pipeline:** Kanban board with applicants by stage. Drag-and-drop to advance, with audit trail.
- **Sourced:** Candidates the recruiter manually added or invited from the talent search.
- **Interviews:** All interviews scheduled for this job.
- **Analytics:** Funnel metrics, time-to-stage, source breakdown, drop-off reasons.
- **Team:** Assignees per stage.
- **Settings:** Edit job, change status, archive.

6.3.4 Talent Search (/talent)

A boolean and natural-language searchable index of public profiles, opted-in candidates, and the company's own private talent pool. Recruiters can save searches as alerts, build talent pools, message candidates directly (with a quota), and bulk-invite to apply to a specific job. The search supports filters mirroring the user job-board filters in reverse (skills, experience, location, education, languages, current company, past companies, industry, availability, salary expectations).

6.4 Candidate Pipeline

6.4.1 Candidate Detail (/candidates/:id)

The complete dossier for a single applicant on a single job. Layout: header with name, photo, contact info, current company, applied date, source, current stage, and quick actions; left tab list (Profile, Resume, Application, Interviews, Notes, Files, Activity, Email); right context panel with AI Match Score breakdown, skill alignment, and red/green flags. Recruiters can advance the candidate, schedule an interview in two clicks, send a message, request a reference, or add internal notes (visible only to the hiring team).

AI Insights on Candidate

- Match score with breakdown (skills, experience, education, location, compensation)
- Resume strengths and weaknesses
- Career-trajectory analysis (rising star, lateral mover, etc.)
- Inferred salary expectation range
- Cultural-fit indicators based on profile and writing style (with bias warnings)
- Suggested interview questions tailored to gaps

6.4.2 Bulk Actions

Multi-select candidates to advance, reject (with optional templated reason and feedback), tag, message, export, or move to a talent pool for future roles. Rejected candidates can be auto-emailed with personalized feedback generated by AI.

6.5 Interviews

6.5.1 Interview Calendar (/interviews)

Calendar view (day, week, month) of every interview across all jobs. Color-coded by job and stage. Click on a slot to view details, reschedule, cancel, or join. Integrates with Google Calendar, Outlook, and Apple Calendar via two-way sync.

6.5.2 Schedule Interview

Two scheduling modes: (1) Direct — recruiter picks a time and the candidate confirms, (2) Self-Scheduling — recruiter sends the candidate a list of slots and the candidate picks one. Slots are derived from the interviewer's connected calendar availability minus blackout times. The system supports panel interviews, sequential interviews (multiple back-to-back rounds with different interviewers), and group interviews (one interviewer, many candidates).

6.5.3 Interview Templates

Reusable interview blueprints. A template specifies duration, format (video, phone, in-person, take-home, live coding), question bank, scorecard rubric, and proctoring settings. Companies can build a library of templates per role family and assign them to pipeline stages.

6.5.4 Live Interview Host (/interview/:id)

The interviewer's view of the proctored live video room. See Section 9. The interviewer sees the candidate's video, their own self-view, a shared whiteboard, a code editor with multi-language support and execution, the question bank with note-taking per question, the structured scorecard, and a discreet proctoring panel showing real-time signals (face presence, eye gaze direction, tab switches, second-person detection). Multiple interviewers can join the same room as a panel. The system can record the session (with consent) and store transcripts and event logs.

6.5.5 Scorecards

After the interview, each interviewer fills a structured scorecard. Rubrics are role-specific and combine quantitative ratings (1-5 scales on dimensions like Technical Depth, Problem Solving, Communication, Culture Add) with qualitative notes. AI assists by drafting a summary from the transcript and notes. All interviewers' scorecards roll up into a hiring decision view, with calibrated scoring to mitigate individual interviewer bias.

6.6 Competitions & Contests

6.6.1 Competitions Manager (/competitions)

Companies create and manage hiring competitions and contests as a top-of-funnel acquisition channel. A competition has a title, description, format (coding challenge, design contest, case study, hackathon, ideathon), eligibility rules, schedule (registration window, submission window, judging window, results date), prize structure (monetary, in-kind, job offers), judging criteria, judges (internal team or external), and submission requirements (code, design files, document, video pitch).

6.6.2 Submissions Review

Reviewer-friendly interface for evaluating entries. Each submission is shown with the candidate's profile, their entry, and a rubric. Judges score independently; the system aggregates and surfaces top submissions. The platform supports anonymous review modes to reduce bias. Winners can be auto-converted into the company's talent pool or directly progressed to interview rounds.

6.7 Company Social & Branding

6.7.1 Company Profile (/profile)

The company's public face on Futurestick. Sections: hero (logo, banner, tagline), about, mission, culture, benefits, life-at gallery (photos, videos), team highlights, jobs, social posts, employee reviews, awards, news. The profile is editable by anyone with the Branding Manager permission.

6.7.2 Posts (/posts)

Companies post to their own feed, which is consumed by followers and surfaces in candidates' main feeds. Post types: announcements, hiring drives, life-at-company, behind-the-scenes, product launches, milestone celebrations, employee spotlights, articles. Scheduling, drafts, and approval workflows (Editor → Approver → Publisher) are supported. Engagement analytics per post are available.

6.7.3 Inbox (/inbox)

Unified inbox for messages from candidates and prospects. Threaded by sender; assignable to teammates; supports canned responses, templates, and auto-reply outside business hours. Integrated with the candidate detail view so a chat from a known applicant deep-links to their dossier.

6.8 Analytics

6.8.1 Hiring Analytics (/analytics)

Multi-dimensional analytics covering the entire hiring funnel. Standard reports include time-to-fill, time-to-hire, source effectiveness, conversion by stage, interviewer scorecard distributions, offer-acceptance rate, candidate experience NPS, diversity metrics (with privacy safeguards), and cost-per-hire (when

integrated with billing). Custom dashboards can be built with a drag-and-drop report builder. Reports can be scheduled and emailed.

6.9 Settings

6.9.1 Workspace Settings

- **Company:** Profile, branding, integrations (HRIS, ATS, calendar, video tools).
- **Team:** Manage members, roles, permissions, departments, audit log.
- **Jobs:** Default templates, pipeline stages, application questions, EEOC settings.
- **Interviews:** Default templates, scorecards, proctoring policies, recording defaults.
- **Notifications:** Slack/email digests, alert thresholds.
- **Billing:** Subscription, seat count, usage, invoices, payment methods.
- **Data & Privacy:** Retention policies, candidate data export/delete, GDPR controls.

7. Admin Panel — Pages and Features

The admin panel is for platform operators (Anthropic-style: Futurestick employees and trusted moderators). It is served at `admin.futurestick.com` and protected by an additional layer of authentication: hardware key required, allowlisted IPs in production, and full audit logging of every action. Admin roles include Super Admin, Platform Admin, Trust & Safety, Finance, Support, and Engineering. Each role has a tightly scoped permission set.

7.1 Admin Dashboard (/dashboard)

Real-time platform health and KPIs: daily active users, weekly active companies, jobs posted, applications, interviews held, AI tasks executed, infrastructure health, error rates, queue depths, payment volume, support ticket backlog. Each tile clicks through to its dedicated module.

7.2 User Management (/users)

Searchable, filterable index of all user accounts. Filters: status, signup date, last active, country, plan, flagged, verified. Per-user actions: view profile, view activity log, suspend, reinstate, force-logout, reset password, anonymize, delete (GDPR), impersonate (with consent and logging), grant/revoke roles. Bulk actions for migrations and clean-ups.

7.3 Company Management (/companies)

Same shape as users. Additional verification queue for companies awaiting approval. Reviewers see incorporation documents, signatory IDs, public web presence, and a fraud-risk score. Decision (approve/reject) is logged with reason. Reverification can be triggered for cause.

7.4 Content Moderation (/moderation)

Queues for moderating reported content: posts, comments, messages, profiles, jobs, company posts, competition submissions. Each item shows the report reason, reporter, content, and AI pre-classification (spam, harassment, hate, NSFW, scam, IP violation, fraud). Moderators can take graduated actions: warn, hide, remove, suspend author, ban, escalate to law enforcement. Decisions feed a continuously trained moderation classifier.

7.5 Jobs Moderation (/jobs)

Index of all jobs platform-wide. Flags suspicious listings: salary-too-good-to-be-true, scam keywords, blacklisted companies, duplicates. Moderators can take down jobs and freeze the posting company pending investigation.

7.6 AI Operations (/ai-ops)

Visibility into AI workloads. Per-feature dashboards: requests/sec, latency, error rate, cost per request, model version in use. A feature-flag system gates new AI capabilities to internal users, then beta cohorts, then general rollout. A prompt registry stores all prompt templates with versioning, A/B test variants, and quality metrics. Model evaluation jobs can be launched against held-out datasets.

7.7 Payments & Billing (/billing)

All payments and subscriptions across users and companies. Reconciliation against the payment processor (Stripe/Razorpay), refund management, dispute handling, invoice generation, tax reporting. Revenue analytics by plan, geography, cohort. Failed-payment retry orchestration.

7.8 Trust, Safety & Compliance (/safety)

Subdomains:

- **Reports:** All user reports, organized by severity.
- **Bans:** Active bans, ban appeals, ban expiration.
- **Fraud:** Suspicious patterns: account farming, mass-applying bots, fake recruiters, payment fraud.
- **Privacy Requests:** GDPR/DPDP/CCPA data export, deletion, correction queues with SLAs.
- **Legal Holds:** Accounts and content under litigation hold cannot be deleted.

7.9 Platform Configuration (/config)

- **Feature Flags:** Boolean and percentage rollouts per feature, per cohort.
- **Skill Taxonomy:** Curated 12,000-skill graph, editable.
- **Industry Taxonomy:** Industries, functions, job-family mappings.
- **Regions & Currencies:** Supported countries, currencies, locales.
- **Email Templates:** Transactional email content per language.
- **Help Center Content:** Articles, FAQs, tutorials.
- **Integrations Catalog:** Available third-party integrations and their statuses.

7.10 Observability (/ops)

Embedded views of Grafana dashboards (latency, error rate, throughput per service), Loki log search, Prometheus alert state, queue depths (BullMQ + Celery), database health (slow queries, replication lag), and a deployment log. Engineers can also trigger maintenance tasks from here: cache invalidation, search reindex, AI model promotion.

7.11 Audit Log (/audit)

Append-only log of every privileged action across all admin tooling. Each entry: actor, action, target, before/after diff, IP, user agent, justification (required for impersonation and data export). Searchable and exportable. Retained for 7 years for compliance.

8. Backend Architecture

8.1 Service Inventory

The backend is composed of fifteen microservices grouped by responsibility and runtime. Node.js handles transactional, real-time, and social workloads; Python handles AI, computer vision, and machine-learning workloads. Every service exposes a stable internal HTTP API (JSON, OpenAPI documented), publishes domain events to a shared message bus (Redis Streams in early stages, Kafka at scale), and emits structured logs and metrics.

Service	Runtime	Primary Responsibility
auth-svc	Node.js	Signup, login, OAuth, JWT issuance, sessions, 2FA, password reset
user-svc	Node.js	Profile CRUD, privacy settings, follow graph, settings
company-svc	Node.js	Company CRUD, team management, verification, branding
job-svc	Node.js	Job CRUD, search, filters, recommendations index
application-svc	Node.js	Applications, pipeline stages, status, screening answers
social-svc	Node.js	Posts, comments, reactions, shares, communities, feed delivery
chat-svc	Node.js	Conversations, messages, presence, typing, read receipts
notification-svc	Node.js	Multi-channel notification orchestration (in-app, email, push, SMS)
interview-svc	Node.js	Scheduling, room provisioning, calendar sync, scorecards
payment-svc	Node.js	Subscriptions, invoicing, payment processing, webhooks
media-svc	Node.js	Pre-signed S3 uploads, image transforms, video transcoding orchestration
ai-orchestrator	Python/FastAPI	Routes AI requests, manages prompts, model selection, rate budgets
ai-resume-svc	Python/FastAPI	Resume parsing, building, enhancing, PDF/DOCX rendering
ai-matcher-svc	Python/FastAPI	Embeddings, similarity, reranking, daily match precomputation
ai-interviewer-svc	Python/FastAPI	AI interviewer (practice), question generation, scoring
ai-counselor-svc	Python/FastAPI	Career counselor agent, knowledge graph, learning paths
proctor-svc	Python/FastAPI	Live computer-vision proctoring, signal aggregation
search-svc	Node.js	Elasticsearch query layer for jobs, people, companies, content

8.2 API Gateway

All client traffic enters through the API gateway, which is implemented in Kong (or NGINX with Lua plugins) deployed behind a cloud load balancer. The gateway terminates TLS, validates the JWT, applies per-route rate limiting, attaches request IDs and trace headers, performs IP geolocation, and routes to the appropriate upstream service. WebSocket upgrades are routed to the chat, social, and interview services. The gateway also handles CORS, response compression, and request body size limits.

8.2.1 Routing Map

```

/api/v1/auth/**          → auth-svc /api/v1/users/**          → user-svc /api/v1/companies/**
→ company-svc /api/v1/jobs/**          → job-svc /api/v1/applications/** → application-svc
/api/v1/posts/**        → social-svc /api/v1/feed/**          → social-svc
/api/v1/communities/**   → social-svc /api/v1/chat/**          → chat-svc
/api/v1/notifications/** → notification-svc /api/v1/interviews/** → interview-svc
/api/v1/payments/**      → payment-svc /api/v1/media/**        → media-svc /api/v1/ai/**
→ ai-orchestrator /api/v1/search/**    → search-svc /ws                      → chat-svc /
social-svc (sticky) /rtc                → interview-svc + mediasoup SFU

```

8.3 Authentication & Authorization

8.3.1 Token Strategy

Futurestick uses a dual-token JWT model. The access token is short-lived (15 minutes), holds the user ID, role, scopes, and panel context, and is sent on every API call as a Bearer header. The refresh token is long-lived (30 days), HTTP-only, secure, SameSite=strict, and never accessible to JavaScript. When the access token expires, the client silently calls `/auth/refresh` to obtain a new access token. Refresh tokens rotate on every use, and the previous token is invalidated; reuse of an old refresh token triggers immediate session revocation across all devices, indicating a likely theft.

8.3.2 RBAC Model

Authorization is role-based with attribute-based extensions. Each user has a global role (USER, COMPANY_MEMBER, ADMIN) plus context roles within companies (OWNER, ADMIN, HIRING_MANAGER, RECRUITER, INTERVIEWER, READ_ONLY). Permissions are expressed as resource:action tuples (e.g., `job:create`, `candidate:view`, `interview:host`). Routes declare required permissions; the auth middleware verifies the JWT, looks up the user's effective permission set (cached in Redis for 60 seconds), and authorizes or rejects. Sensitive admin actions require step-up authentication via WebAuthn at the moment of action.

8.3.3 OAuth Providers

Google, LinkedIn, GitHub, Microsoft are supported. The OAuth flow uses PKCE on the client. On callback, if the email matches an existing account, the OAuth identity is linked (with explicit user confirmation); otherwise a new account is created and prefilled from OAuth claims. LinkedIn-sourced accounts can additionally pull profile, experience, and education to seed the Futurestick profile.

8.4 Inter-Service Communication

8.4.1 Synchronous (HTTP)

Services communicate over internal HTTP using a shared SDK auto-generated from each service's OpenAPI spec. Calls carry the original request ID and trace headers for end-to-end tracing. Internal HTTP uses mTLS, rotated automatically by cert-manager. Calls have aggressive defaults: 2-second timeout, 1 retry on 502/503/504, circuit breaker tripping after 50% error rate over 30 seconds. Sensitive cross-service operations (e.g., payment to user upgrade) are idempotent and use deterministic IDs.

8.4.2 Asynchronous (Events)

Services publish domain events to Redis Streams (or Kafka in production). Event types are versioned and registered in a shared schema registry. Consumers are at-least-once with idempotent handlers. Examples:

```

user.created          → matcher (build embeddings), notification (welcome email)
user.profile.updated  → matcher (refresh embeddings), search (reindex)
job.created           → matcher (precompute matches), search (index)
application.submitted → notification (recruiter), analytics (funnel event)
application.advanced  → notification (candidate),
interview-svc (offer slots)
interview.completed   → ai (transcribe + score), analytics
post.created          → feed (fanout), search (index), moderation (classify)
payment.succeeded     → user-svc (entitlement), notification (receipt)

```

8.5 API Endpoint Catalog

The full OpenAPI specification is generated from each service's source and lives at `/docs/api`. The following catalog summarizes the most important endpoints by domain. Every endpoint follows REST conventions: GET for reads, POST for creates, PATCH for partial updates, PUT for replacements, DELETE for removals. List endpoints support cursor-based pagination (cursor, limit), filtering (`?filter[field]=value`), sorting (`?sort=-createdAt`), and sparse fieldsets (`?fields=...`).

8.5.1 Auth

```
POST /auth/signup POST /auth/login POST /auth/logout POST /auth/refresh POST
/auth/oauth/:provider POST /auth/forgot-password POST /auth/reset-password POST
/auth/verify-email POST /auth/2fa/enroll POST /auth/2fa/verify GET /auth/sessions
DELETE /auth/sessions/:id
```

8.5.2 Users & Profiles

```
GET /users/me PATCH /users/me GET /users/:handle GET /users/:id/profile PATCH
/users/me/profile PUT /users/me/profile/sections/:section POST /users/me/skills DELETE
/users/me/skills/:id POST /users/:id/follow DELETE /users/:id/follow POST
/users/:id/connect POST /connections/:id/accept GET /users/me/connections GET
/users/me/settings PATCH /users/me/settings
```

8.5.3 Companies

```
POST /companies GET /companies/:id PATCH /companies/:id POST /companies/:id/members
PATCH /companies/:id/members/:userId DELETE /companies/:id/members/:userId POST
/companies/:id/follow GET /companies/:id/jobs GET /companies/:id/posts POST
/companies/:id/verify GET /companies/:id/analytics
```

8.5.4 Jobs

```
POST /jobs GET /jobs?filters GET /jobs/:id PATCH /jobs/:id DELETE /jobs/:id POST
/jobs/:id/save POST /jobs/:id/apply GET /jobs/:id/applicants PATCH /jobs/:id/pipeline
GET /jobs/recommended GET /jobs/:id/analytics
```

8.5.5 Applications

```
POST /applications GET /applications/me GET /applications/:id PATCH
/applications/:id/stage POST /applications/:id/withdraw POST /applications/:id/notes GET
/applications/:id/timeline
```

8.5.6 Social

```
POST /posts GET /posts/:id PATCH /posts/:id DELETE /posts/:id POST
/posts/:id/reactions POST /posts/:id/comments GET /posts/:id/comments POST
/posts/:id/share GET /feed?cursor&filter GET /communities POST /communities POST
/communities/:id/join GET /communities/:id/feed
```

8.5.7 Chat

```
GET /conversations POST /conversations GET /conversations/:id/messages POST
/conversations/:id/messages PATCH /conversations/:id/read POST /conversations/:id/typing
(over WS) POST /conversations/:id/mute POST /conversations/:id/archive
```

8.5.8 Interviews

```

POST /interviews GET /interviews/:id PATCH /interviews/:id POST /interviews/:id/cancel
POST /interviews/:id/reschedule POST /interviews/:id/start (issues room token)
POST /interviews/:id/end POST /interviews/:id/scorecard GET /interviews/:id/recording
GET /interviews/:id/proctor-events GET /interviews/:id/transcript

```

8.5.9 AI

```

POST /ai/resume/parse POST /ai/resume/build POST /ai/resume/enhance POST
/ai/resume/score POST /ai/cover-letter/generate POST /ai/match/score GET
/ai/match/daily POST /ai/auto-apply/run POST /ai/counselor/chat POST
/ai/interviewer/start POST /ai/interviewer/turn POST /ai/interviewer/finish POST
/ai/skill-gap/analyze POST /ai/negotiator/draft

```

9. AI Modules — Detailed Specifications

AI is woven throughout Futurestick. This section specifies each AI module: its inputs, outputs, model selection strategy, prompt or pipeline shape, evaluation method, latency target, and failure-handling approach.

9.1 AI Orchestrator

A central FastAPI service that fronts every AI feature. It manages prompt templates (versioned in a registry), selects the underlying model based on feature, language, and cost budget, applies rate limits and quota tracking per user and per company, captures every input and output to an evaluation store (with PII redaction), and exposes a unified streaming API to clients. The orchestrator supports OpenAI, Anthropic, Google Gemini, and self-hosted open-source models (via Ollama or vLLM) interchangeably; switching providers is a feature-flag change.

9.2 Resume Parser

Converts an uploaded resume (PDF or DOCX) into structured JSON conforming to the Futurestick profile schema. Pipeline: (1) text extraction with PyMuPDF and python-docx, fallback to OCR via Tesseract for scanned PDFs, (2) layout-aware section detection using a fine-tuned transformer that classifies each text block (header, contact, summary, experience, education, skills, etc.), (3) entity extraction within each section using few-shot LLM prompts, (4) normalization (dates, locations, skills mapped to taxonomy), (5) confidence scoring per field. Fields below 0.6 confidence are flagged for user review. Target latency: under 8 seconds for a 2-page resume.

9.3 Resume Builder

Generates resumes from the user's structured profile and a chosen template. The template is a Jinja-rendered HTML structure with CSS, converted to PDF by WeasyPrint. The builder also offers AI-assist within each section: bullet-point rewrites that strengthen action verbs, add quantification, and align with target keywords. The user supplies a job description (optional) and the builder uses a constrained generation strategy that preserves factual content (no hallucinated achievements) while improving phrasing.

9.4 Resume Enhancer

Multi-pass critique. Pass 1 — ATS analyzer (deterministic): keyword extraction, font and layout checks, tabular detection, file-size and image-count checks. Pass 2 — LLM critique: clarity, impact, redundancy, action-verb diversity, quantification gaps, with a per-bullet 0-100 score and rewrite suggestion. Pass 3 — Job-specific alignment (if a JD is provided): keyword match score, missing critical skills, suggested reordering. The output is a side-by-side diff with one-click accept/reject per change.

9.5 AI Job Matcher

The matcher combines vector similarity with a learned reranker.

- **Stage 1 — Embeddings:** Each user's profile and each job description are encoded as 768-dim sentence-transformer embeddings (mxbai-embed-large or similar). User embeddings are recomputed on profile update; job embeddings on job edit. Stored in pgvector or Pinecone.
- **Stage 2 — ANN Retrieval:** For a given user, the top 500 most similar jobs are retrieved via approximate nearest neighbor search.
- **Stage 3 — Reranker:** A gradient-boosted tree (XGBoost) reranks the 500 candidates using explicit features: cosine similarity, location distance, salary alignment, work-mode match, experience bracket match, language overlap, education match, skill-graph proximity, recency, and historical engagement (clicks, applies, dwell time). Trained on platform engagement data.
- **Stage 4 — Diversity:** Maximal Marginal Relevance is applied to ensure the final 20 are not all from the same company or role.
- **Stage 5 — Explanation:** An LLM generates a short, plain-English rationale for each surfaced job: which strengths matter and which gaps are minor.

9.6 AI Auto-Apply

A consent-bound agent. The user defines: target roles, locations, work mode, salary minimum, daily quota (default 5), and the minimum match score (default 80). A nightly Celery job retrieves matched jobs above threshold, generates a tailored resume per job, an AI-written cover letter, and answers screening questions from the user's profile. The user reviews each application in a queue at /ai/auto-apply by default, approving or rejecting; experienced users can enable full autonomy with mandatory daily summary emails. Every applied job is logged in the user's application list with an "AI-applied" badge for transparency.

9.7 AI Career Counselor

A LangGraph-orchestrated agent. It maintains a running conversation, has access to tools (profile reader, job searcher, salary benchmarker, course recommender, skill-graph traverser, market-trend reporter), and a long-term memory of the user's stated goals, constraints, and prior decisions (stored in a per-user vector store). The counselor uses a chain-of-thought style reasoning loop: Plan → Use Tool → Observe → Reflect → Respond. It refuses to make decisions for the user and always presents trade-offs.

9.8 AI Interviewer (Practice)

A real-time conversational simulator. Architecture:

- **Question Selector:** Given role, seniority, and chosen mode, samples a question from a curated bank or generates a novel one. Adapts based on prior answers ("You handled the design question well, let's go deeper").
- **Speech In:** OpenAI Whisper (or local equivalent) transcribes the user's voice in real time. Confidence and disfluency markers retained for scoring.
- **Reasoning:** An LLM evaluates the answer against a rubric (content, structure, examples, depth, clarity), decides whether to ask a follow-up, then generates the next prompt.
- **Speech Out:** Coqui TTS (or ElevenLabs in cloud mode) renders the next question with the chosen interviewer voice.
- **Scoring:** At session end, the LLM produces per-question scores, an overall report with strengths, gaps, and study recommendations.

9.9 AI Cover Letter Generator

A simpler module. Inputs: profile, job, optional tone and length. The system uses a structured prompt with retrieval over the user's most relevant experiences, projects, and achievements, then generates a draft. The output is editable rich text. The user can regenerate sections without losing accepted ones.

9.10 AI Skill Gap Analyzer

Compares the user's skill set against the union of skills required by their target roles or a specific job. For each missing or weak skill, the analyzer pulls courses from the partner catalog, weights by relevance and rating, estimates time-to-proficiency, and orders by leverage (impact per hour invested). Output: a 3- to 12-month learning roadmap.

9.11 AI Salary Negotiator

Inputs: current offer, user's market position (computed from their profile and platform data), competing offers (if disclosed), priorities. The model drafts a written negotiation message and a verbal script. A voice rehearsal mode lets the user practice with the AI playing the recruiter, with feedback after each turn.

9.12 AI Moderation

A multi-class classifier runs on every user-generated content item (post, comment, message, job description, profile field, competition entry). Categories: spam, hate, harassment, NSFW, scam, IP infringement, fraud, self-harm. Above-threshold items are routed to the moderation queue; high-confidence severe items (e.g., CSAM signals via PhotoDNA) are auto-removed and reported. Every decision is logged for appeal.

10. Live Interview Engine

10.1 Goals

Futurestick's built-in live interview engine eliminates the need for third-party video tools (Zoom, Google Meet) by combining low-latency video, role-aware tooling (whiteboard, code editor), and a robust proctoring layer in a single, integrated experience. The engine is engineered to feel as natural as a video call while continuously producing structured signal that helps interviewers evaluate candidates fairly and detect dishonest behavior. All proctoring is opt-in, transparent, and explained to the candidate before the session starts.

10.2 Media Architecture

The video plane is built on WebRTC with a Selective Forwarding Unit (mediasoup). For one-on-one interviews, the system can fall back to peer-to-peer to minimize bandwidth and latency; for panel interviews and recorded sessions, all media flows through the SFU. STUN servers (Google public + redundant) enable host discovery; coturn TURN servers enable relay through restrictive NATs. Media tracks are encrypted end-to-end with insertable streams; the SFU sees only opaque bytes.

10.2.1 Signaling Flow

```
1. Candidate clicks "Join" → POST /interviews/:id/start
2. interview-svc validates: time window, identity, consent
3. interview-svc returns: room token, SFU URL, ICE servers
4. Client connects to SFU via WebSocket (signaling)
5. SFU authenticates room token (HMAC-signed)
6. Client publishes audio/video/screen tracks
7. SFU forwards to other participants based on room state
8. Proctoring tracks (video frames sampled) → proctor-svc via WebSocket
9. Periodic event publishes (transcripts, signals) to interview-svc
10. On end → finalization job: transcript, recording mux, scorecard prefill
```

10.3 Proctoring Subsystem

Proctoring runs entirely client-side for sensitive vision (no raw video is sent to a proctoring server unless the candidate explicitly consents to recording). The browser samples webcam frames at 4-6 FPS into an off-screen canvas, runs lightweight models in WebGL/WASM via MediaPipe Tasks, and emits compact event messages (booleans, angles, IDs) to the proctor service. The service aggregates events into per-minute summaries and a cumulative integrity score, which is shown to interviewers in real time.

10.3.1 Proctoring Signals

- **Face Presence:** Is exactly one face detected? Continuous absence beyond 3 seconds is a flag.
- **Face Identity:** Periodic face-embedding comparison against the candidate's verified identity photo. Drift beyond a similarity threshold triggers a flag (possible candidate swap).
- **Multi-Person Detection:** More than one face in frame for more than 2 seconds is a flag (possible coaching).

- **Eye Gaze Tracking:** MediaPipe Face Mesh + iris landmarks estimate gaze vector. Sustained off-screen gaze (looking at second monitor or notes) is flagged.
- **Head Pose:** Pitch/yaw/roll estimation. Repeated extreme deviations (looking down at phone, behind monitor) are flagged.
- **Mouth Activity vs Speech:** Detects when the candidate's mouth is closed but speech is heard (someone else talking) or vice versa (lip-sync issues).
- **Tab Switch Detection:** Page Visibility API + focus events detect when the browser tab is no longer active. Each occurrence and duration logged.
- **Window Blur:** Window-level focus loss (alt-tab to another app).
- **Copy/Paste Activity:** Clipboard events captured to detect pasting answers.
- **Network Quality:** RTT, jitter, packet loss; correlated with anomalies (a glitch may explain a flagged event).
- **Audio Anomaly:** Background voices, keyboard clatter, room echo via simple audio classification.
- **Device Sanity:** Virtual camera detection (heuristics on device name, image variance). Flag if a virtual camera is in use.
- **Browser Sanity:** DevTools open detection, headless browser markers, automation flags.
- **Environment Scan (Optional):** At session start, candidate is asked to slowly pan camera around the room; the recording is saved as evidence.
- **Screen Share:** For coding rounds, full-screen share is required and analyzed for unexpected window changes.

10.3.2 Integrity Score

Each signal contributes a weighted deduction to a starting score of 100. Weights are tunable per company policy. The final score is a continuous 0-100 plus a categorical recommendation (Clean, Minor, Significant, Severe). The score is never the sole basis for a hiring decision; it is a flag that prompts the interviewer to ask follow-up questions or, in extreme cases, to invalidate the session.

10.4 Tools Inside the Room

- **Whiteboard:** Excalidraw-based collaborative drawing. Multi-cursor, shape libraries, export as image.
- **Code Editor:** Monaco editor with multi-language support (JS, TS, Python, Java, C++, Go, Rust, SQL, etc.), ESLint/Prettier integration, run button executing code in a sandboxed Docker runner via the AI service. Includes test-case panel for live evaluation.
- **Question Bank:** Interviewer can pin pre-loaded questions from the template; per-question note-taking, timer, and rubric scoring inline.
- **File Share:** Drag-and-drop documents (e.g., a problem statement PDF). Auto-deleted post-interview unless saved.
- **Chat:** Sidebar text chat for sharing links and clarifications without disrupting voice flow.

- **Transcript:** Live captions via Whisper streaming, with speaker diarization. Searchable, exportable.

10.5 Recording & Storage

Recording is opt-in and explicitly consented to by the candidate before joining. When enabled, the SFU server-side records each track separately at the highest quality, then post-processes: mixed audio + video MP4 (for review), separate per-track WebM (for reanalysis), full transcript with timestamps, proctor-event timeline, and AI-generated summary. Recordings are encrypted at rest with KMS, retained per company policy (default 90 days, configurable), and access-logged.

10.6 AI-Generated Interview Summary

Within minutes of the interview ending, the system delivers a structured summary to interviewers: an executive overview, per-question scoring with quoted candidate excerpts, identified strengths and concerns, suggested follow-up questions for the next round, and a sentiment/communication analysis. This summary populates the interviewer's scorecard as a draft, which they review and edit before submitting.

10.7 Network Resilience

Lossy networks are common in many regions. The engine adapts: bitrate is auto-reduced to as low as 100 kbps for video and 16 kbps for audio; if video fails entirely, the call falls back to audio-only with a candidate avatar; if both fail, the session pauses with a buffered "reconnecting" state for up to 60 seconds before officially marking a disruption. All disruptions are logged and visible in the post-interview report, ensuring the candidate is not penalized for network issues.

11. Database Schemas

11.1 Storage Strategy Recap

Futurestick uses a polyglot persistence model. MySQL stores transactional, relational entities where ACID and joins matter — accounts, companies, jobs, applications, interviews, payments. MongoDB stores high-cardinality, evolving, or document-shaped data — posts, comments, chat, AI artifacts, feed cache, interview event timelines. Redis is the fast-path layer for sessions, hot caches, rate limits, queues, and pub/sub. Elasticsearch indexes searchable text. All schemas below specify primary keys, foreign keys (where applicable), indices, and notes on partitioning.

11.2 MySQL Schemas

11.2.1 users

```
CREATE TABLE users ( id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT, uuid
CHAR(36) NOT NULL UNIQUE, handle VARCHAR(64) NOT NULL UNIQUE, email
VARCHAR(255) NOT NULL UNIQUE, email_verified_at DATETIME NULL, phone
VARCHAR(32) NULL, phone_verified_at DATETIME NULL, password_hash VARCHAR(255) NULL,
-- null for OAuth-only first_name VARCHAR(100) NOT NULL, last_name
VARCHAR(100) NOT NULL, avatar_url VARCHAR(512) NULL, cover_url VARCHAR(512)
NULL, headline VARCHAR(220) NULL, about TEXT NULL, location_city
VARCHAR(120) NULL, location_country CHAR(2) NULL, timezone VARCHAR(64) NULL,
locale VARCHAR(16) DEFAULT 'en', status
ENUM('active','suspended','deleted') DEFAULT 'active', role
ENUM('user','company_member','admin') DEFAULT 'user', open_to_work BOOLEAN DEFAULT
FALSE, two_fa_enabled BOOLEAN DEFAULT FALSE, last_active_at DATETIME NULL,
created_at DATETIME DEFAULT CURRENT_TIMESTAMP, updated_at DATETIME DEFAULT
CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, INDEX idx_email (email), INDEX idx_handle
(handle), INDEX idx_location (location_country, location_city), INDEX idx_open_to_work
(open_to_work, last_active_at) );
```

11.2.2 user_oauth_identities

```
CREATE TABLE user_oauth_identities ( id BIGINT UNSIGNED PRIMARY KEY
AUTO_INCREMENT, user_id BIGINT UNSIGNED NOT NULL, provider
ENUM('google','linkedin','github','microsoft') NOT NULL, provider_uid VARCHAR(255) NOT
NULL, email VARCHAR(255) NULL, raw_profile JSON NULL, linked_at DATETIME
DEFAULT CURRENT_TIMESTAMP, UNIQUE KEY uniq_provider (provider, provider_uid), INDEX
idx_user (user_id), FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE );
```

11.2.3 user_profile_sections

Section data common to all users. Specialized sections (experience, education) have dedicated tables for queryability.

```
CREATE TABLE user_experiences ( id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
user_id BIGINT UNSIGNED NOT NULL, company_id BIGINT UNSIGNED NULL, -- linked
if a known company company_name VARCHAR(200) NOT NULL, title VARCHAR(200) NOT NULL,
employment_type ENUM('full_time','part_time','contract','intern','freelance') NULL,
work_mode ENUM('onsite','remote','hybrid') NULL, location VARCHAR(200) NULL,
start_date DATE NOT NULL, end_date DATE NULL, -- null means current
```

```

description TEXT NULL, skills JSON NULL, created_at DATETIME DEFAULT
CURRENT_TIMESTAMP, updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP, INDEX idx_user (user_id, start_date DESC), FOREIGN KEY (user_id)
REFERENCES users(id) ON DELETE CASCADE ); CREATE TABLE user_educations ( id
BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT, user_id BIGINT UNSIGNED NOT NULL,
institution VARCHAR(200) NOT NULL, degree VARCHAR(120) NULL, field
VARCHAR(200) NULL, grade VARCHAR(60) NULL, start_date DATE NULL, end_date
DATE NULL, description TEXT NULL, INDEX idx_user (user_id), FOREIGN KEY (user_id)
REFERENCES users(id) ON DELETE CASCADE ); CREATE TABLE user_skills ( id BIGINT
UNSIGNED PRIMARY KEY AUTO_INCREMENT, user_id BIGINT UNSIGNED NOT NULL, skill_id
BIGINT UNSIGNED NOT NULL, proficiency ENUM('beginner','intermediate','advanced','expert')
DEFAULT 'intermediate', years DECIMAL(4,1) NULL, endorsements INT DEFAULT 0,
is_featured BOOLEAN DEFAULT FALSE, UNIQUE KEY uniq_user_skill (user_id, skill_id), INDEX
idx_skill (skill_id) ); CREATE TABLE skills ( id BIGINT UNSIGNED PRIMARY KEY
AUTO_INCREMENT, name VARCHAR(200) NOT NULL UNIQUE, slug VARCHAR(220) NOT NULL
UNIQUE, type ENUM('hard','soft','tool','language','domain') NOT NULL, parent_id BIGINT
UNSIGNED NULL, INDEX idx_parent (parent_id) );

```

11.2.4 companies

```
CREATE TABLE companies ( id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT, uuid
CHAR(36) NOT NULL UNIQUE, slug VARCHAR(120) NOT NULL UNIQUE, legal_name
VARCHAR(255) NOT NULL, brand_name VARCHAR(255) NOT NULL, website VARCHAR(255)
NULL, logo_url VARCHAR(512) NULL, cover_url VARCHAR(512) NULL, industry_id
BIGINT UNSIGNED NULL, size ENUM('1-10','11-50','51-200','201-500','501-
1000','1001-5000','5000+') NULL, founded_year SMALLINT UNSIGNED NULL, headquarters
VARCHAR(255) NULL, country CHAR(2) NULL, about TEXT NULL, mission
TEXT NULL, culture TEXT NULL, verification_status
ENUM('pending','verified','rejected','suspended') DEFAULT 'pending', verified_at
DATETIME NULL, status ENUM('active','paused','deleted') DEFAULT 'active', plan
ENUM('free','starter','growth','enterprise') DEFAULT 'free', followers_count INT DEFAULT 0,
created_at DATETIME DEFAULT CURRENT_TIMESTAMP, updated_at DATETIME DEFAULT
CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, INDEX idx_slug (slug), INDEX idx_industry
(industry_id), INDEX idx_status (status, verification_status) ); CREATE TABLE
company_members ( id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT, company_id
BIGINT UNSIGNED NOT NULL, user_id BIGINT UNSIGNED NOT NULL, role
ENUM('owner','admin','hiring_manager','recruiter','interviewer','read_only') NOT NULL,
department VARCHAR(120) NULL, invited_by BIGINT UNSIGNED NULL, joined_at DATETIME
DEFAULT CURRENT_TIMESTAMP, status ENUM('invited','active','suspended','removed')
DEFAULT 'active', UNIQUE KEY uniq_company_user (company_id, user_id), INDEX idx_user
(user_id), FOREIGN KEY (company_id) REFERENCES companies(id) ON DELETE CASCADE, FOREIGN
KEY (user_id) REFERENCES users(id) ON DELETE CASCADE );
```

11.2.5 jobs

```
CREATE TABLE jobs ( id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT, uuid
CHAR(36) NOT NULL UNIQUE, company_id BIGINT UNSIGNED NOT NULL, posted_by BIGINT
UNSIGNED NOT NULL, title VARCHAR(220) NOT NULL, department VARCHAR(120)
NULL, description MEDIUMTEXT NOT NULL, responsibilities MEDIUMTEXT NULL,
requirements MEDIUMTEXT NULL, benefits MEDIUMTEXT NULL, employment_type
ENUM('full_time','part_time','contract','intern','freelance') NOT NULL, work_mode
ENUM('onsite','remote','hybrid') NOT NULL, experience_level
ENUM('entry','mid','senior','lead','executive') NOT NULL, experience_years_min DECIMAL(4,1)
NULL, experience_years_max DECIMAL(4,1) NULL, salary_currency CHAR(3) NULL, salary_min
DECIMAL(12,2) NULL, salary_max DECIMAL(12,2) NULL, salary_period
ENUM('hour','month','year') NULL, salary_visible BOOLEAN DEFAULT TRUE, equity_offered
BOOLEAN DEFAULT FALSE, visa_sponsored BOOLEAN DEFAULT FALSE, headcount INT DEFAULT
1, application_deadline DATE NULL, status
ENUM('draft','active','paused','closed','archived') DEFAULT 'draft', views_count INT
DEFAULT 0, applies_count INT DEFAULT 0, posted_at DATETIME NULL, closed_at
DATETIME NULL, created_at DATETIME DEFAULT CURRENT_TIMESTAMP, updated_at
DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, INDEX idx_company
(company_id, status, posted_at DESC), INDEX idx_status_posted (status, posted_at DESC),
INDEX idx_experience (experience_level), INDEX idx_work_mode (work_mode), FOREIGN KEY
(company_id) REFERENCES companies(id) ON DELETE CASCADE ); CREATE TABLE job_locations ( id
BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT, job_id BIGINT UNSIGNED NOT NULL, city
VARCHAR(120) NULL, region VARCHAR(120) NULL, country CHAR(2) NOT NULL, is_primary
BOOLEAN DEFAULT FALSE, INDEX idx_job (job_id), INDEX idx_country_city (country, city),
FOREIGN KEY (job_id) REFERENCES jobs(id) ON DELETE CASCADE ); CREATE TABLE job_skills (
job_id BIGINT UNSIGNED NOT NULL, skill_id BIGINT UNSIGNED NOT NULL, is_required BOOLEAN
DEFAULT TRUE, weight DECIMAL(3,2) DEFAULT 1.0, PRIMARY KEY (job_id, skill_id), INDEX
idx_skill (skill_id) ); CREATE TABLE job_screening_questions ( id BIGINT UNSIGNED
PRIMARY KEY AUTO_INCREMENT, job_id BIGINT UNSIGNED NOT NULL, question TEXT NOT NULL,
type ENUM('text','multiselect','number','yesno','file') NOT NULL, options JSON NULL,
```

```
required BOOLEAN DEFAULT FALSE,   knockout BOOLEAN DEFAULT FALSE,   knockout_rule JSON NULL,  
display_order INT NOT NULL,   FOREIGN KEY (job_id) REFERENCES jobs(id) ON DELETE CASCADE );
```

11.2.6 applications

```
CREATE TABLE applications ( id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  uuid CHAR(36) NOT NULL UNIQUE, job_id BIGINT UNSIGNED NOT NULL,
  user_id BIGINT UNSIGNED NOT NULL, resume_url VARCHAR(512) NOT NULL,
  cover_letter MEDIUMTEXT NULL, source
  ENUM('direct','ai_auto','referral','recruiter_invite','community','external') DEFAULT
  'direct', current_stage VARCHAR(64) NOT NULL DEFAULT 'applied', match_score
  DECIMAL(5,2) NULL, ai_assisted BOOLEAN DEFAULT FALSE, status
  ENUM('active','rejected','withdrawn','offered','hired') DEFAULT 'active', rejection_reason
  VARCHAR(255) NULL, applied_at DATETIME DEFAULT CURRENT_TIMESTAMP, decided_at
  DATETIME NULL, UNIQUE KEY uniq_user_job (user_id, job_id), INDEX idx_job_stage (job_id,
  current_stage), INDEX idx_user (user_id, applied_at DESC), FOREIGN KEY (job_id) REFERENCES
  jobs(id) ON DELETE CASCADE, FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE );
CREATE TABLE application_stage_history ( id BIGINT UNSIGNED PRIMARY KEY
  AUTO_INCREMENT, application_id BIGINT UNSIGNED NOT NULL, from_stage VARCHAR(64) NULL,
  to_stage VARCHAR(64) NOT NULL, changed_by BIGINT UNSIGNED NULL, note TEXT
  NULL, changed_at DATETIME DEFAULT CURRENT_TIMESTAMP, INDEX idx_app (application_id,
  changed_at), FOREIGN KEY (application_id) REFERENCES applications(id) ON DELETE CASCADE );
CREATE TABLE application_answers ( id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  application_id BIGINT UNSIGNED NOT NULL, question_id BIGINT UNSIGNED NOT NULL, answer
  JSON NOT NULL, FOREIGN KEY (application_id) REFERENCES applications(id) ON DELETE CASCADE );
```

11.2.7 interviews

```
CREATE TABLE interviews ( id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT, uuid
  CHAR(36) NOT NULL UNIQUE, application_id BIGINT UNSIGNED NOT NULL, job_id BIGINT
  UNSIGNED NOT NULL, candidate_id BIGINT UNSIGNED NOT NULL, company_id BIGINT
  UNSIGNED NOT NULL, template_id BIGINT UNSIGNED NULL, title VARCHAR(200) NOT
  NULL, format ENUM('video','phone','onsite','take_home') DEFAULT 'video',
  scheduled_at DATETIME NOT NULL, duration_minutes INT NOT NULL DEFAULT 60, timezone
  VARCHAR(64) NOT NULL, status
  ENUM('scheduled','live','completed','cancelled','no_show','rescheduled') DEFAULT 'scheduled',
  room_id VARCHAR(64) NULL, recording_consent BOOLEAN DEFAULT FALSE,
  proctoring_enabled BOOLEAN DEFAULT TRUE, recording_url VARCHAR(512) NULL, transcript_url
  VARCHAR(512) NULL, integrity_score DECIMAL(5,2) NULL, started_at DATETIME NULL,
  ended_at DATETIME NULL, created_at DATETIME DEFAULT CURRENT_TIMESTAMP, INDEX
  idx_candidate (candidate_id, scheduled_at DESC), INDEX idx_company (company_id, scheduled_at
  DESC), INDEX idx_status (status, scheduled_at), FOREIGN KEY (application_id) REFERENCES
  applications(id) ON DELETE CASCADE ); CREATE TABLE interview_participants ( id
  BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT, interview_id BIGINT UNSIGNED NOT NULL,
  user_id BIGINT UNSIGNED NOT NULL, role
  ENUM('candidate','interviewer','observer') NOT NULL, joined_at DATETIME NULL, left_at
  DATETIME NULL, FOREIGN KEY (interview_id) REFERENCES interviews(id) ON DELETE CASCADE );
CREATE TABLE interview_scorecards ( id BIGINT UNSIGNED PRIMARY KEY
  AUTO_INCREMENT, interview_id BIGINT UNSIGNED NOT NULL, interviewer_id BIGINT UNSIGNED NOT
  NULL, overall_rating DECIMAL(3,1) NOT NULL, recommendation
  ENUM('strong_yes','yes','neutral','no','strong_no') NOT NULL, scores JSON NOT NULL,
  -- per-rubric-dimension scores strengths TEXT NULL, concerns TEXT NULL, notes
  MEDIUMTEXT NULL, submitted_at DATETIME DEFAULT CURRENT_TIMESTAMP, UNIQUE KEY
  uniq_interviewer (interview_id, interviewer_id), FOREIGN KEY (interview_id) REFERENCES
  interviews(id) ON DELETE CASCADE );
```

11.2.8 competitions

```
CREATE TABLE competitions ( id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT, uuid
CHAR(36) NOT NULL UNIQUE, slug VARCHAR(160) NOT NULL UNIQUE, company_id BIGINT
UNSIGNED NOT NULL, title VARCHAR(220) NOT NULL, description MEDIUMTEXT NOT NULL,
format ENUM('coding','design','case_study','hackathon','ideathon','other') NOT NULL,
registration_starts DATETIME NOT NULL, registration_ends DATETIME NOT NULL,
submission_starts DATETIME NOT NULL, submission_ends DATETIME NOT NULL, judging_ends
DATETIME NOT NULL, results_at DATETIME NULL, prize_pool DECIMAL(12,2)
NULL, prize_currency CHAR(3) NULL, prize_breakdown JSON NULL, eligibility
JSON NULL, status
ENUM('draft','registration_open','submission_open','judging','closed','cancelled') DEFAULT
'draft', visibility ENUM('public','private','invite_only') DEFAULT 'public',
created_at DATETIME DEFAULT CURRENT_TIMESTAMP, INDEX idx_status (status,
registration_ends), FOREIGN KEY (company_id) REFERENCES companies(id) ON DELETE CASCADE );
CREATE TABLE competition_registrations ( id BIGINT UNSIGNED PRIMARY KEY
AUTO_INCREMENT, competition_id BIGINT UNSIGNED NOT NULL, user_id BIGINT UNSIGNED
NOT NULL, team_id BIGINT UNSIGNED NULL, registered_at DATETIME DEFAULT
CURRENT_TIMESTAMP, UNIQUE KEY uniq_competition_user (competition_id, user_id) ); CREATE
TABLE competition_submissions ( id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
competition_id BIGINT UNSIGNED NOT NULL, user_id BIGINT UNSIGNED NOT NULL,
team_id BIGINT UNSIGNED NULL, files JSON NOT NULL, notes TEXT
NULL, submitted_at DATETIME DEFAULT CURRENT_TIMESTAMP, score DECIMAL(5,2)
NULL, rank INT NULL, is_winner BOOLEAN DEFAULT FALSE );
```

11.2.9 payments

```
CREATE TABLE subscriptions ( id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
owner_type ENUM('user','company') NOT NULL, owner_id BIGINT UNSIGNED NOT NULL,
plan VARCHAR(64) NOT NULL, status
ENUM('trial','active','past_due','cancelled','expired') NOT NULL, provider
ENUM('stripe','razorpay','paddle') NOT NULL, provider_sub_id VARCHAR(160) NOT NULL,
current_period_start DATETIME NOT NULL, current_period_end DATETIME NOT NULL,
cancel_at_period_end BOOLEAN DEFAULT FALSE, INDEX idx_owner (owner_type, owner_id), INDEX
idx_status (status) ); CREATE TABLE invoices ( id BIGINT UNSIGNED PRIMARY KEY
AUTO_INCREMENT, subscription_id BIGINT UNSIGNED NULL, owner_type ENUM('user','company')
NOT NULL, owner_id BIGINT UNSIGNED NOT NULL, amount DECIMAL(12,2) NOT NULL,
currency CHAR(3) NOT NULL, tax DECIMAL(12,2) DEFAULT 0, status
ENUM('draft','open','paid','void','uncollectible') NOT NULL, pdf_url VARCHAR(512)
NULL, issued_at DATETIME NOT NULL, due_at DATETIME NULL, paid_at
DATETIME NULL, INDEX idx_owner (owner_type, owner_id, issued_at DESC) );
```

11.2.10 connections, follows, notifications

```
CREATE TABLE follows ( id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
follower_id BIGINT UNSIGNED NOT NULL, followee_type ENUM('user','company') NOT NULL,
followee_id BIGINT UNSIGNED NOT NULL, created_at DATETIME DEFAULT
CURRENT_TIMESTAMP, UNIQUE KEY uniq_follow (follower_id, followee_type, followee_id), INDEX
idx_followee (followee_type, followee_id) ); CREATE TABLE connections ( id
BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT, requester_id BIGINT UNSIGNED NOT NULL,
recipient_id BIGINT UNSIGNED NOT NULL, status
ENUM('pending','accepted','declined','blocked') NOT NULL, message VARCHAR(500) NULL,
created_at DATETIME DEFAULT CURRENT_TIMESTAMP, responded_at DATETIME NULL, UNIQUE
KEY uniq_pair (requester_id, recipient_id), INDEX idx_recipient (recipient_id, status) );
CREATE TABLE notifications ( id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT, user_id
BIGINT UNSIGNED NOT NULL, type VARCHAR(64) NOT NULL, payload JSON NOT NULL,
```



```
is_read    BOOLEAN DEFAULT FALSE,    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,    INDEX  
idx_user_unread (user_id, is_read, created_at DESC) );
```

11.3 MongoDB Collections

Document stores host the high-volume, evolving content. All collections include createdAt and updatedAt fields, indexes on user/company foreign keys, and time-based indexes where queries are recency-driven. Document IDs use UUID v7 for time-orderable primary keys.

11.3.1 posts

```
{ _id: UUID,  authorType: 'user' | 'company',  authorId: Long,  type: 'update' |
'looking_for_job' | 'showcase' | 'question' | 'announcement' | 'job_promo' |
'competition_promo',  visibility: 'public' | 'connections' | 'community' |
'company_followers',  communityId: UUID | null,  body: { text: String, mentions: [{ id:
Long, type: String, range: [n,n] }], tags: [String] },  media: [ { type: 'image' |
'video' | 'document' | 'link',  url: String, thumbnail: String, width: Int, height: Int,
alt: String, durationMs: Int } ],  poll: { question: String, options: [String], multiple:
Bool, closesAt: Date } | null,  lookingForJob: {  targetRoles: [String], locations:
[String], workMode: String,  noticePeriodDays: Int, expectedSalary: { min, max, currency },
skills: [String] } | null,  metrics: { reactions: Int, comments: Int, shares: Int, views:
Int, saves: Int },  reactionBreakdown: { like: Int, celebrate: Int, insightful: Int, support:
Int, funny: Int, curious: Int },  isEdited: Bool,  isPinned: Bool,  moderation: { status:
'clean'|'pending'|'flagged'|'removed', reasons: [String] },  createdAt: Date,  updatedAt:
Date } // Indexes { authorType: 1, authorId: 1, createdAt: -1 } { communityId: 1, createdAt:
-1 } { "moderation.status": 1, createdAt: -1 } { type: 1, createdAt: -1 } { "body.tags": 1 }
```

11.3.2 comments

```
{ _id: UUID,  postId: UUID,  parentId: UUID | null,  authorId: Long,  authorType: 'user'
| 'company',  body: { text: String, mentions: [...] },  reactionCount: Int,
reactionBreakdown: {...},  replyCount: Int,  createdAt: Date,  updatedAt: Date,
deletedAt: Date | null } // Indexes { postId: 1, createdAt: 1 } { parentId: 1, createdAt: 1 }
{ authorId: 1, createdAt: -1 }
```

11.3.3 reactions

```
{ _id: UUID,  targetType: 'post' | 'comment',  targetId: UUID,  userId: Long,  reaction:
'like' | 'celebrate' | 'insightful' | 'support' | 'funny' | 'curious',  createdAt: Date } //
Indexes { targetType: 1, targetId: 1, userId: 1 } unique { userId: 1, createdAt: -1 }
```

11.3.4 conversations & messages

```
// conversations { _id: UUID,  type: 'direct' | 'group' | 'support' | 'company_inbox',
participants: [{ id: Long, role: String, joinedAt: Date }],  title: String | null,
lastMessage: { id: UUID, senderId: Long, preview: String, sentAt: Date },  pinned: [UUID],
archivedFor: [Long],  mutedFor: [{ userId: Long, until: Date | null }],  unread: { [userId]:
Int },  createdAt: Date,  updatedAt: Date } { "participants.id": 1, updatedAt: -1 } //
messages (one collection per conversation? we keep one collection sharded by conversationId) {
_id: UUID,  conversationId: UUID,  senderId: Long,  body: { text: String, mentions: [...]
},  attachments: [{ type, url, name, size }],  replyTo: UUID | null,  reactions: [{ userId:
Long, emoji: String }],  readBy: [{ userId: Long, readAt: Date }],  isEdited: Bool,
deletedAt: Date | null,  sentAt: Date } { conversationId: 1, sentAt: -1 } { senderId: 1,
sentAt: -1 }
```

11.3.5 feed_index

Materialized fan-out feed for users with many followees. Hybrid push-pull: high-follower-count authors are pulled at read time; others fan out at write time.

```
{ _id: UUID,  userId: Long,  postId: UUID,  postedAt: Date,  source: 'follow' |
'community' | 'recommendation' | 'sponsored',  score: Double } { userId: 1, score: -1,
postedAt: -1 }
```

11.3.6 ai_artifacts

```
{ _id: UUID,  ownerId: Long,  ownerType: 'user' | 'company',  feature: 'resume' |
'cover_letter' | 'enhancement' | 'match_explanation' | 'interview_summary' |
'counselor_session',  inputs: { ... },          // redacted PII  outputs: { ... },
modelMeta: { provider, model, version, tokensIn, tokensOut, latencyMs, costUsd },  feedback:
{ rating: Int, helpful: Bool, comments: String },  createdAt: Date } { ownerType: 1, ownerId:
1, feature: 1, createdAt: -1 }
```

11.3.7 interview_events

Granular event log per live interview. Used for proctor-event timeline, replay, and post-hoc audits.

```
{ _id: UUID,  interviewId: Long,  participantId: Long,  ts: Date,  kind: 'face_lost' |
'multi_face' | 'gaze_off' | 'tab_blur' | 'window_blur' | 'paste' |          'speech' |
'silence' | 'network_drop' | 'recording_started' | 'question_asked' |          'code_run' |
'whiteboard_draw' | 'screen_share_start' | 'integrity_flag',  payload: { ... } } // Time-
series collection (MongoDB 5+) timeseries: { timeField: 'ts', metaField: { interviewId,
participantId } }
```

11.3.8 communities

```
{ _id: UUID,  slug: String,  name: String,  description: String,  visibility: 'public' |
'request' | 'invite',  cover: String,  rules: [String],  topics: [String],  members: {
count: Int, sample: [Long] },  moderators: [Long],  createdBy: Long,  status: 'active' |
'suspended',  createdAt: Date }
```

11.4 Redis Keyspaces

- **session:{userId}:{sessionId}**: Active session metadata, TTL = refresh-token lifetime
- **ratelimit:{route}:{ip}**: Sliding-window counters
- **cache:user:{id}**: Hot user profile, TTL 60s
- **cache:job:{id}**: Hot job, TTL 120s
- **feed:user:{id}**: Cached feed page, TTL 30s
- **presence:{userId}**: Online status, TTL 60s, refreshed by heartbeat
- **typing:{conversationId}**: Set of currently typing user IDs, TTL 5s
- **queue:bull:***: BullMQ queues for jobs, emails, notifications
- **stream:events**: Redis Stream for domain events
- **pubsub:socketio***: Socket.IO Redis adapter pub/sub

11.5 Elasticsearch Indices

- **jobs**: Indexed on title, description, skills, location, salary, company. Refreshed via DB CDC. Synonyms file maintained for skill aliases.
- **users**: Indexed on name, headline, skills, experience text. Privacy-respecting: only public fields.

- **companies:** Indexed on brand_name, about, industry.
- **posts:** Indexed on body, tags, author. Used for content search and trending discovery.

12. Security & Compliance

12.1 Threat Model

Futurestick handles personal data (resumes, salary expectations, interview recordings, identity documents), financial data (subscriptions, payouts), and potentially sensitive HR decisions. The threat model considers: account takeover (credential stuffing, phishing), privilege escalation (user → admin), data exfiltration (insider, leaked tokens, scraping), tampering with hiring decisions, fake recruiters or fake candidates (fraud), interview proctoring evasion, denial of service, and supply-chain attacks on dependencies.

12.2 Authentication Hardening

- **Password Policy:** Minimum 10 chars, mix recommended, checked against haveibeenpwned k-anonymity API on signup.
- **Hashing:** Argon2id (m=64MB, t=3, p=2) or bcrypt cost 12; pepper stored in KMS.
- **2FA:** TOTP (RFC 6238) with backup codes; WebAuthn for high-trust roles.
- **Suspicious Login Detection:** Geo-velocity, device fingerprinting, novel-IP step-up.
- **Session Revocation:** Server-side revocation list keyed by JTI; refresh-token reuse triggers cascade.
- **Brute Force:** Progressive delay + CAPTCHA + IP block.

12.3 Authorization

Every API route declares (1) authentication mode (anonymous / authenticated / company-scoped / admin), (2) required permissions, (3) resource ownership rules. The middleware first verifies the JWT, then checks permissions in the user's effective set, then evaluates row-level access ("this candidate belongs to this company's hiring team"). Permission checks are also re-applied at the data layer: queries always include scope filters to prevent leaks via mistakenly broad service code.

12.4 Data Protection

- **In Transit:** TLS 1.3 everywhere, HSTS, certificate pinning for mobile clients, mTLS for inter-service.
- **At Rest:** AES-256-GCM via cloud KMS for databases, S3 SSE-KMS for media, envelope encryption for sensitive columns (e.g., user phone, identity doc URLs).
- **Field-Level Encryption:** PII fields like phone, government ID numbers, and salary expectations are encrypted with a per-tenant key.
- **Backups:** Encrypted, geographically separated, restoration tested monthly.
- **Secrets:** Vault with dynamic database credentials, short-lived; never in env files outside dev.

12.5 Privacy & Compliance

- **GDPR / DPDP / CCPA:** Self-serve data export and deletion. SLA: 30 days. Subject rights tracked in admin queue.
- **Data Minimization:** Profile fields are optional; AI features run on minimum necessary data; recordings retained per company-set policy.
- **Consent:** Granular consent for: marketing, AI processing, recordings, third-party sharing. Revocable anytime.
- **Data Residency:** EU users' data stored in EU region clusters; configurable for enterprise customers.
- **Audit Logs:** All admin actions, data exports, and impersonations logged for 7 years.
- **DPA & Sub-processors List:** Public, kept current.

12.6 Application Security

- **Input Validation:** Zod (TS) and Pydantic (Py) schemas on every endpoint.
- **Output Encoding:** React escaping by default, manual review for dangerouslySetInnerHTML.
- **CSRF:** SameSite=strict cookies + CSRF tokens for non-idempotent calls.
- **CSP:** Strict CSP with nonce-based scripts, no unsafe-inline.
- **CORS:** Explicit allowlist per panel.
- **Rate Limiting:** Per-user, per-IP, per-route at gateway.
- **File Uploads:** MIME sniffing, virus scan via ClamAV, size caps, signed direct-to-S3 with policy constraints.
- **SSRF Protection:** URL allowlists for outbound calls (link previews, OAuth).
- **Dependency Scanning:** Dependabot + Snyk + license scanning in CI.
- **Secrets Scanning:** GitGuardian on every commit.
- **SAST/DAST:** CodeQL on PRs; OWASP ZAP weekly scan.
- **Bug Bounty:** Public program with HackerOne/Bugcrowd at scale.

12.7 Anti-Abuse

- **Bot Detection:** Cloudflare Turnstile / hCaptcha on signup and high-risk actions.
- **Spam Detection:** Per-action heuristics + ML classifier on posts/messages.
- **Fake Recruiters:** Company verification, signal-based scoring, suspension on confirmed reports.
- **Mass-Apply Bots:** Quotas, behavioral signals, captcha triggers.
- **Scraping Defense:** Edge bot management, signed pagination cursors, structured-data limits.

13. Real-Time Layer & Messaging

13.1 Socket.IO Topology

Real-time delivery is centralized in Socket.IO clusters with the Redis adapter. Each Socket.IO node is stateless; sticky load balancing is enforced at the gateway via cookie-based affinity. Rooms are namespaced per feature: /chat, /feed, /interview, /notifications. Message delivery is at-least-once; idempotency on the client de-duplicates by event ID.

13.2 Notification Pipeline

Domain Event → notification-svc rules engine → channel selection → in-app (write to MySQL + push via Socket.IO) → email (BullMQ queue → SES with template) → push (FCM/APNs for installed apps) → SMS (Twilio for high-priority only) User preferences gate channels per category. Digest emails coalesce low-priority notifications.

13.3 Chat Delivery

Sending a message follows: client → POST /conversations/:id/messages → chat-svc validates and writes to MongoDB → publishes message.created → Socket.IO fan-out to participants in the room. Read receipts and typing are ephemeral, never persisted past 30 seconds. End-to-end encryption is opt-in for direct chats; the server holds only ciphertext, with key exchange via signal-protocol-style ratchet.

13.4 Feed Fan-Out

New posts are pushed to followers using a hybrid model. For authors with under 10,000 followers, the post is fanned out into each follower's feed_index at write time. For high-cardinality authors (companies with millions of followers, viral users), the post stays in a global by-author index and is pulled into followers' feeds at read time. The transition threshold and read-time merge are tunable.

14. Public Landing Page

14.1 Goals

The marketing site at futurestick.com is the platform's first impression. It must be fast, beautiful, persuasive, and SEO-optimized. It is intentionally built as plain HTML, CSS, and vanilla JavaScript (with optional GSAP for animations) rather than React, because: (a) static HTML loads instantly and ranks better on Core Web Vitals, (b) marketers can edit copy without a build step, (c) the site never needs the application's dependency tree, (d) crawlers parse it without JS execution.

14.2 Page Inventory

- **Home (/):** Hero, feature highlights, social proof, CTAs to sign up.
- **For Job Seekers (/for-job-seekers):** Detailed walkthrough of user-side features.
- **For Companies (/for-companies):** Detailed walkthrough of employer-side features.
- **AI Features (/ai):** Showcase of resume builder, matcher, counselor, interviewer.
- **Live Interview (/live-interview):** Deep dive into the proctoring engine, with demo video.
- **Competitions (/competitions):** How contests work, prize ladder, partner showcase.
- **Pricing (/pricing):** Tiered plans for users and companies.
- **Customers (/customers):** Logos, case studies, testimonials.
- **Partners (/partners):** Educational institutes, training providers.
- **Blog (/blog):** Static-rendered articles via Markdown + 11ty.
- **Help (/help):** Searchable knowledge base.
- **Legal (/legal/*):** Terms, Privacy, DPA, Cookies, Acceptable Use.
- **Contact (/contact):** Form + offices.
- **Careers (/careers):** Open jobs at Futurestick (eats own dog food via the platform).

14.3 Folder Structure

```

landing/ |— public/                                # Final static output |— src/ | |— pages/
# 11ty templates per page | |— _layouts/ | |— _components/ | |— data/                                #
CMS data files (JSON/YAML) | |— styles/ | | |— reset.css | | |— tokens.css                                #
Design tokens shared with app design system | | |— components.css | | |— pages/ |
|— scripts/ | | |— nav.js | | |— animations.js # GSAP scenes | | |—
analytics.js | | |— forms.js | |— assets/ |— eleventy.config.js |— package.json

```

14.4 Performance Targets

- LCP under 1.8 s on 3G
- CLS under 0.05
- TBT under 100 ms

- Total JS under 50 KB gzipped
- Total CSS under 30 KB gzipped
- Images served as AVIF + WebP fallback, served via image CDN
- Critical CSS inlined; rest deferred
- Fonts subset and self-hosted

14.5 SEO

- Per-page title, meta description, canonical, OG, Twitter card
- JSON-LD: Organization, Product, FAQ, BreadcrumbList, JobPosting (for promoted listings)
- XML sitemap auto-generated; robots.txt curated
- Hreflang tags for multilingual versions
- Structured internal linking

15. Infrastructure & Deployment

15.1 Environments

- **Local:** Docker Compose; isolated MySQL, MongoDB, Redis, MinIO; LocalStack for AWS service mocks.
- **Dev:** Shared Kubernetes cluster, low-resource, ephemeral databases reset weekly.
- **Staging:** Production-mirroring cluster, anonymized production data, full third-party integrations in sandbox mode.
- **Production:** Multi-AZ, auto-scaling, blue-green deploys for critical services, canaries for risky changes.

15.2 Cluster Layout

Production runs on Kubernetes (EKS or GKE). Namespaces separate concerns: ingress, app-node, app-python, ai, data-proxies, observability, security. Each microservice is a Deployment behind a ClusterIP Service, exposed externally through the gateway only. Secrets are mounted via External Secrets Operator backed by Vault. ConfigMaps hold non-sensitive configuration. Horizontal Pod Autoscalers scale by CPU and request rate; AI services additionally scale on queue depth.

15.3 Datastore Topology

- **MySQL:** Primary + 2 read replicas. Daily snapshots, point-in-time recovery via binlog. Schema migrations via Prisma Migrate gated by manual approval in production.
- **MongoDB:** Replica set (1 primary, 2 secondaries) with automatic failover; sharded clusters for chat and feed_index at scale.
- **Redis:** Multi-AZ Redis Cluster (6 nodes, 3 shards × replica). Separate clusters for sessions, queues, and pub/sub to avoid noisy-neighbor effects.
- **Elasticsearch:** 3-node cluster with dedicated master nodes; daily snapshots to S3.
- **S3:** Buckets per concern: user-uploads, resumes, recordings, public-assets, backups. Lifecycle rules transition cold data to Glacier.

15.4 CI/CD

GitHub Actions runs CI: lint, type-check, unit tests, integration tests, dependency scan, secrets scan, and build. On merge to main, container images are built (multi-stage Dockerfiles, distroless base) and pushed to a private registry. ArgoCD watches a deployment manifest repo and rolls out changes via GitOps. Database migrations run as Kubernetes Jobs gated on a successful preflight job that diff-checks the proposed schema change.

15.5 Observability

- **Logs:** Loki, structured JSON, retention 30 days hot + 1 year cold.
- **Metrics:** Prometheus, retention 15 days; long-term in Mimir/Thanos.
- **Traces:** OpenTelemetry collector → Tempo / Jaeger; sampling 1-10% adaptive.
- **Errors:** Sentry for frontend and backend, with release tracking.
- **Synthetic Monitoring:** Checkly probes for critical user journeys every 60 seconds.
- **Alerts:** PagerDuty integration with severity routing.

15.6 Disaster Recovery

- RPO target: 5 minutes
- RTO target: 1 hour for read paths, 4 hours for full write recovery
- Quarterly DR drills with documented runbooks
- Cross-region backup replication for tier-1 data

16. Testing Strategy

16.1 Test Pyramid

- **Unit:** Per-function/component tests in Jest (Node), Vitest (frontend), Pytest (Python). Coverage target 80%.
- **Component:** React components tested in isolation with React Testing Library.
- **Contract:** Pact tests verify API contracts between services.
- **Integration:** Service-level tests with real MySQL/Mongo/Redis in Docker Compose.
- **End-to-End:** Playwright suites covering critical user journeys per panel; runs on every PR + nightly full suite.
- **Visual:** Chromatic visual regression on the design system and key pages.
- **Load:** k6 scripts for chat, feed, and interview joining throughput.
- **Security:** OWASP ZAP weekly + targeted pen-test annually.
- **AI Evaluations:** Per-feature held-out datasets, golden answers, regression checks; runs nightly with quality dashboards.

16.2 Critical User Journeys (E2E)

- User signup → onboarding → first feed view → first job apply
- Company signup → verification → first job post → first applicant review
- AI resume build → enhance → export to PDF
- Schedule interview → join interview → proctoring active → submit scorecard
- Direct message thread → file share → video escalation
- Competition create → register → submit → judge → results
- Subscription upgrade → invoice → entitlement change
- Admin moderation: flag content → review → action → audit log

17. Implementation Roadmap

17.1 Phasing Philosophy

Futurestick is a large platform; implementation is split into shippable phases that each deliver standalone value while building toward the full vision. Each phase ends with a usable product, even if narrower in scope. AI features ship gradually, often behind feature flags and starting with admins/beta cohorts before general rollout.

17.2 Phase 1 — Foundation (Weeks 1-8)

- Auth, user profiles, company profiles
- Job posting and basic job board with search and filters
- Application submission and basic pipeline
- Landing page and onboarding flows
- Admin panel scaffolding and verification queue

17.3 Phase 2 — Social Layer (Weeks 9-14)

- Posts, comments, reactions, follows, connections
- Personalized feed (chronological + simple ranking)
- Communities (basic)
- Notifications system
- Direct chat (text-only)

17.4 Phase 3 — AI Tools (Weeks 15-22)

- Resume parser + builder + enhancer
- AI Job Matcher with explanations
- AI Cover Letter Generator
- AI Career Counselor (chat)
- Skill Gap Analyzer

17.5 Phase 4 — Live Interviews (Weeks 23-30)

- WebRTC + SFU integration
- Interview scheduling + calendar sync
- Whiteboard, code editor, transcript
- Proctoring v1: face presence, gaze, multi-person, tab-switch
- Recording + AI summary

- Scorecards + structured rubrics

17.6 Phase 5 — Competitions & Marketplace (Weeks 31-36)

- Competition creation, registration, submission, judging
- Mentor marketplace
- Learning hub with partner content
- Auto-Apply (consent-based)
- AI Interviewer (practice mode)

17.7 Phase 6 — Scale & Hardening (Weeks 37-44)

- Sharding for chat and feed
- ES → ANN search migration
- End-to-end encryption for chat (opt-in)
- Multi-region deployment
- Mobile apps via React Native or PWA
- Public bug bounty
- ISO 27001 / SOC 2 readiness

17.8 Phase 7 — Continuous Enhancement

- Advanced proctoring (audio analysis, virtual cam detection)
- Salary Negotiator AI
- Real-time AI assistance during user interviews (where consented)
- Marketplace for paid services (resume reviews, mentorship)
- Internationalization to 10+ languages

18. Appendices

18.1 Glossary

- **ATS:** Applicant Tracking System — software used by employers to manage applications.
- **RBAC:** Role-Based Access Control.
- **SFU:** Selective Forwarding Unit — server that routes WebRTC media to multiple peers.
- **STUN/TURN:** Servers that help WebRTC traverse NATs.
- **WebRTC:** Browser-native real-time communication standard.
- **SPA:** Single-Page Application.
- **SSR / SSG:** Server-Side Rendering / Static Site Generation.
- **LCP / CLS / TBT:** Core Web Vitals: Largest Contentful Paint, Cumulative Layout Shift, Total Blocking Time.
- **PII:** Personally Identifiable Information.
- **KMS:** Key Management Service.
- **CDC:** Change Data Capture — propagating DB changes to other systems.
- **ANN:** Approximate Nearest Neighbor search, used for vector similarity.
- **MMR:** Maximal Marginal Relevance — balances similarity with diversity in result lists.
- **OWASP:** Open Web Application Security Project — security guidelines and tools.
- **SLA / SLO / SLI:** Service Level Agreement / Objective / Indicator.
- **DPA:** Data Processing Agreement, required under GDPR.
- **MFA / 2FA:** Multi-Factor / Two-Factor Authentication.
- **WebAuthn:** Public-key-based passwordless authentication standard.

18.2 Naming Conventions

- URLs: kebab-case (/job-board, /resume-studio)
- Database tables: snake_case, plural (users, job_skills)
- API JSON keys: camelCase
- Internal events: dot notation (post.created, application.advanced)
- Code symbols: PascalCase classes/components, camelCase functions/variables

18.3 Versioning

- APIs: /v1, /v2 in URL path; deprecation cycle 12 months
- Mobile apps: forced upgrade after 6 months; soft prompt earlier
- Database migrations: forward-compatible, two-phase for breaking changes

18.4 Document Index Summary

- **Sections 1-2:** Overview & Architecture
- **Section 3:** Tech Stack
- **Section 4:** Frontend Architecture
- **Section 5:** User Panel — Pages & Features
- **Section 6:** Company Panel — Pages & Features
- **Section 7:** Admin Panel — Pages & Features
- **Section 8:** Backend Architecture
- **Section 9:** AI Modules
- **Section 10:** Live Interview Engine
- **Section 11:** Database Schemas
- **Section 12:** Security & Compliance
- **Section 13:** Real-Time Layer
- **Section 14:** Public Landing Page
- **Section 15:** Infrastructure & Deployment
- **Section 16:** Testing Strategy
- **Section 17:** Implementation Roadmap
- **Section 18:** Appendices

— End of Document —